



Indian Institute of Information Technology Guwahati

Department of Computer Science and Engineering

Fast Authentication Between UAVs

A PUF-Based Four-Phase Swarm Authentication Protocol:
Design and Provable Security

Author: Md Ziyad Hussain

Roll No.: 2301128

Email: ziyad.hussain23b@iiitg.ac.in

Supervisor: Dr. Rakesh Matam
Assistant Professor, CSE

Indian Institute of Information Technology Guwahati

Bongora, Assam 781015, India

2026

Abstract

Unmanned Aerial Vehicle (UAV) swarms require authentication that is fast, lightweight, and resilient to the physical capture of individual drones. Traditional public-key infrastructure (RSA-2048, ECC-256) adds millisecond-scale handshake cost on embedded processors and, more seriously, relies on private keys stored in non-volatile memory—a liability the moment a UAV is captured. We present a four-phase authentication protocol whose root of trust is a Physical Unclonable Function (PUF) embedded in each UAV. The protocol covers (1) secure enrollment, in which a fuzzy extractor over a BCH(255,131,18) code turns noisy PUF measurements into a *stable* per-device master key while storing no secret on the drone; (2) UAV–Ground Station (GS) mutual authentication in which every message is protected by a keyed message-authentication code and the raw PUF response never leaves the device; (3) direct UAV–UAV peer authentication, without GS involvement, rooted in *per-pair* credentials delivered under authenticated encryption; and (4) session-key establishment in which a mandatory, authenticated ephemeral Diffie–Hellman exchange provides perfect forward secrecy. The protocol uses only a 160-bit hash—instantiable as SHA3-256 truncated to 160 bits or SPONGENT-160—together with a keyed MAC, a lightweight authenticated-encryption scheme, a key-derivation function, and a single elliptic curve, so that it fits comfortably on resource-constrained aerial platforms. We give a complete security analysis: six theorems covering mutual authentication in both directions, peer authentication, replay and man-in-the-middle resistance, physical-capture resistance, session-key secrecy, and perfect forward secrecy, each reduced to standard assumptions and mirrored by a symbolic model in the Tamarin prover. A reference implementation in OMNeT++ 6.3 authenticates all 600 UAVs and all 11,400 peer pairs of a twenty-drone swarm across thirty repetitions, at a simulated elapsed time of 1.50 ± 0.002 ms per UAV–GS handshake and 0.417 ± 0.001 ms per peer pair, with every primitive checked against published reference values. Executed attacks confirm the analysis: forged ground-station messages draw no response, whereas the same adversary succeeds against a control variant with the ground-station check removed, so the negative result is falsifiable. Measured on the same host as RSA-2048 and elliptic-curve baselines, the ephemeral key exchange this protocol mandates for forward secrecy costs about as much as a bare ECDH/ECDSA exchange, not an order of magnitude less. A full same-platform handshake-to-handshake comparison against real RSA-authenticated and ECDSA-authenticated ephemeral-DH protocols places this work at 2.68 ms against 2.05 ms (RSA) and 0.83 ms (ECDSA): closer to RSA than a primitive-level reading alone would suggest, and genuinely slower than ECDSA, the disclosed cost of storing no long-term secret on the device. This protocol’s advantages are properties rather than raw speed: no stored long-term secret, no certificate infrastructure, GS-free peer authentication, and capture damage bounded to a node’s own links.

Keywords: UAV authentication, Physical Unclonable Function, PUF, SPONGENT, SHA3, swarm security, lightweight cryptography, peer-to-peer authentication, fuzzy extractor, BCH code, forward secrecy.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Limitations of Traditional Authentication	4
1.3	Physical Unclonable Functions as a Solution	4
1.4	Contributions of This Paper	4
2	Background and Existing Technologies	5
2.1	UAV Communication Architecture	5
2.2	Physical Unclonable Functions (PUFs)	5
2.2.1	PUF Fundamentals	5
2.2.2	Types of PUFs	5
2.2.3	PUF Noise and Reliability	5
2.3	Lightweight Cryptographic Hash Functions	5
2.3.1	SHA3-256 (Keccak)	5
2.3.2	SPONGENT-160	5
2.4	Fuzzy Extractors and BCH Error Correction	5
2.4.1	Fuzzy Extractors	5
2.4.2	BCH Code Parameters	6
3	Related Work	6
3.1	PUF-Based Authentication for IoT and UAVs	6
3.2	Peer-to-Peer UAV Authentication	6
3.3	Lightweight Hash Functions for Constrained Devices	6
3.4	Comparison of Existing Approaches	6
4	Proposed Protocol Design	7
4.1	Threat Model and Design Goals	7
4.1.1	Threat Model	7
4.1.2	Design Goals	7
4.2	System Model and Notation	7
4.3	Phase 1: Secure Enrollment and Registration	8
4.4	Phase 2: UAV–Ground Station Mutual Authentication	9
4.5	Phase 3: UAV-to-UAV Peer Authentication	11
4.6	Phase 4: Session Key Establishment and Forward Secrecy	12
5	Security Analysis	12
5.1	Assumptions	12
5.2	Phase 2: Mutual Authentication	13
5.3	Replay Attack Resistance	13
5.4	Phase 3: Peer Mutual Authentication	13
5.5	Man-in-the-Middle Resistance	14
5.6	Session-Key Secrecy and Forward Secrecy	14
5.7	Physical Capture Resistance	14
5.8	Symbolic Verification	14
5.9	Summary of Security Properties	15

6	Reference Implementation	15
6.1	Structure	15
6.2	Canonical message encoding	15
6.3	Cryptographic suites	16
6.4	Error-correcting code	16
6.5	Fuzzy-extractor parameterisation	17
6.6	PUF models	17
6.7	Verification of primitives	18
6.8	Simulation model	18
6.9	Evaluation scenarios	18
7	Evaluation	20
7.1	Methodology	20
7.2	Functional correctness	20
7.3	Latency and communication cost	20
7.4	Message-by-message breakdown	21
7.5	Phase-3 message-by-message breakdown	22
7.6	Primitive Costs: This Work versus Public-Key Baselines	22
7.7	Full-protocol comparison against RSA/ECDSA baselines	24
7.8	Reliability of the fuzzy extractor	25
7.9	Security evaluation	26
7.10	UAV mobility	27
7.11	Real 802.11 MAC contention	28
7.12	Energy cost	28
8	Discussion	29
8.1	Scalability	29
8.2	Design Choices	29
8.3	Noise Tolerance	30
8.4	Limitations and Future Work	30
9	Conclusion	30

1 Introduction

1.1 Background and Motivation

Unmanned Aerial Vehicles (UAVs) have become integral to modern defense, disaster response, precision agriculture, environmental monitoring, and public safety. Multi-UAV *swarms*—ten or more drones cooperating to surveil a stadium, patrol a border, or search a disaster zone—are now the standard operational unit [1]. Such swarms communicate continuously with a Ground Station (GS) over a command-and-control link and with one another over peer links to maintain formation, fuse sensor data, and route around outages.

A fundamental prerequisite for any of this is *mutual authentication*: each UAV must verify the identity of its GS and of every peer before accepting commands, sharing telemetry, or relaying messages. Without it, an adversary can inject a rogue drone into the swarm, intercept sensitive data, issue unauthorized flight commands, or mount man-in-the-middle attacks on inter-UAV channels [9].

1.2 Limitations of Traditional Authentication

Conventional options fit the UAV-swarm regime poorly. **RSA-2048 PKI** requires several milliseconds per signature verification on embedded processors and attaches hundreds of bytes of certificate per handshake [2]. **ECC-256** halves the cost but still keeps a private key in non-volatile memory (NVM) that a captured drone can surrender through side-channel or cold-boot attacks [3]. **Pre-shared keys** reach sub-millisecond latency but compromise the entire swarm if any single device is captured, and require $O(N^2)$ pairwise keys [8]. **Blockchain-based** schemes incur seconds of consensus latency, incompatible with real-time coordination [4]. In every case the weakness is the same: a long-term secret *stored* on a device that may fall into an adversary’s hands.

1.3 Physical Unclonable Functions as a Solution

A Physical Unclonable Function (PUF) is a hardware primitive that maps challenges to responses using intrinsic, sub-nanometer manufacturing variations unique to one integrated circuit [10]. A PUF is *unclonable* (it cannot be replicated, even by its manufacturer), *unpredictable* (responses to unseen challenges are unrelated to prior ones), *tamper-evident* (invasive probing destroys the response), and *lightweight* (an Arbiter PUF needs roughly 1,200 gate equivalents). Crucially, *no key is stored*: the “key” is the silicon itself. This removes the dominant attack surface of PKI and PSK schemes and is the foundation of our design.

1.4 Contributions of This Paper

We present a complete four-phase PUF-based authentication framework for UAV swarms. Its contributions are:

1. **A stable PUF-derived key with no stored secret.** Enrollment distills a stable per-device master key from the PUF via a fuzzy extractor; the drone regenerates it on demand from its PUF and *public* helper data, so no secret ever resides in NVM.
2. **Keyed authentication that keeps the PUF response on-device.** Every handshake message is authenticated with a keyed MAC under a PUF-derived key, and the raw PUF response is never transmitted, so it cannot be observed or harvested over the air.
3. **GS-free peer authentication with per-pair trust.** Peers authenticate directly using *per-pair* credentials delivered under authenticated encryption during Phase 2, so a compromised drone endangers only its own links.
4. **Mandatory forward secrecy.** An authenticated ephemeral Diffie–Hellman exchange is folded into every session key, so a later compromise of long-term material cannot unlock past sessions.
5. **A complete security analysis.** Six theorems, each reduced to standard cryptographic assumptions and corroborated by a mechanised Tamarin symbolic model, establish the protocol’s authentication, secrecy, and capture-resistance guarantees.

The remainder of the paper covers background (§2), related work (§3), the protocol design (§4), the security analysis (§5), the reference implementation (§6), the evaluation (§7), a discussion of

scalability and design choices (§8), and the conclusion (§9).

2 Background and Existing Technologies

2.1 UAV Communication Architecture

A UAV swarm has two communication planes. The *vertical* plane connects each UAV to the GS for commands, telemetry, and mission updates; the *horizontal* plane connects UAVs to one another for formation control and data relay. Authentication must secure both, and it must continue to work on the horizontal plane even when the GS link is briefly unavailable. Our design therefore treats UAV–GS and UAV–UAV authentication as distinct phases with distinct trust roots.

2.2 Physical Unclonable Functions (PUFs)

2.2.1 PUF Fundamentals

A PUF implements a challenge–response mapping $R = \text{PUF}(C)$ that is determined by uncontrollable fabrication variations. Two nominally identical chips produce statistically independent responses to the same challenge (inter-chip Hamming distance near 50%), while a single chip reproduces its own response with only small deviation (intra-chip Hamming distance of a few percent). These two properties—uniqueness and reliability—make a PUF a per-device fingerprint that no attacker can duplicate.

2.2.2 Types of PUFs

Common strong PUFs include the *Arbiter PUF*, which races two symmetric delay paths and outputs a bit according to which signal arrives first, and the *Ring-Oscillator PUF*, which compares oscillator frequencies. Strong PUFs expose an exponentially large challenge space, which is what allows a PUF to serve as a source of many independent secrets. We assume a strong PUF with a 128-bit challenge space throughout.

2.2.3 PUF Noise and Reliability

Because a PUF measures physical quantities, its response drifts with temperature, supply voltage, and aging, producing a bit-error rate (BER) of typically 3–10% between evaluations of the same challenge. A raw response therefore cannot be used directly as a key; it must first be reconciled to a stable value. This is the role of the fuzzy extractor (§2.4).

2.3 Lightweight Cryptographic Hash Functions

2.3.1 SHA3-256 (Keccak)

SHA3-256 is the NIST-standard sponge hash [20, 21]. Truncated to 160 bits it offers 80-bit collision resistance and benefits from fast, widely available software implementations, making it our software-oriented instantiation of the abstract hash $h(\cdot)$.

2.3.2 SPONGENT-160

SPONGENT-160 is a lightweight sponge hash designed for hardware, occupying only a few thousand gate equivalents [15]. It shares the 160-bit interface of truncated SHA3, so the protocol can switch between the two without any structural change; SPONGENT is our hardware-oriented instantiation.

2.4 Fuzzy Extractors and BCH Error Correction

2.4.1 Fuzzy Extractors

A fuzzy extractor turns a noisy PUF response into a reproducible key using public *helper data* [5]. It is a pair of algorithms (Gen, Rep). Given an enrollment response R , Gen produces helper data P and a key; given a later noisy response R' and the same P , Rep recovers the identical key whenever R' is close enough to R . We use the standard *code-offset* construction over a linear code C : enrollment picks a random codeword c and publishes the offset $s = R \oplus c$; reproduction computes $c' = R' \oplus s$ and decodes it back to c , so that if the Hamming distance $d_H(R, R') \leq t$ the original response R is recovered exactly.

An important and often-misstated point concerns how much the public helper data reveals. The offset s does *not* perfectly hide R ; it discloses a bounded amount of information—at most the code’s

redundancy, $n - k$ bits. The correct security statement is therefore about *residual min-entropy*: if R has min-entropy m , then R retains at least $m - (n - k)$ bits of min-entropy given P . A strong extractor applied to R then yields a key that is statistically close to uniform, provided the residual entropy comfortably exceeds the key length. We make this precise in Lemma 1 and use it to size the enrollment in Phase 1.

2.4.2 BCH Code Parameters

We instantiate the code with BCH(255,131,18) over $\text{GF}(2^8)$:

- **Codeword length** $n = 255$ bits;
- **Message length** $k = 131$ bits;
- **Error-correction capability** $t = 18$ bit-flips;
- **Minimum distance** $d = 37$;
- **Redundancy** $n - k = 124$ parity bits.

At a 3% error rate a 128-bit response is expected to flip $128 \times 0.03 \approx 3.84$ bits, well within the $t = 18$ correction radius, so reproduction succeeds with overwhelming probability. Decoding uses the standard pipeline of syndrome computation, the Berlekamp–Massey algorithm to find the error-locator polynomial, a Chien search to locate errors, and bit-flipping to correct them [6].

3 Related Work

3.1 PUF-Based Authentication for IoT and UAVs

Early PUF authentication schemes target a single device authenticating to a server [2, 8, 13]. They establish the PUF-plus-fuzzy-extractor pattern but do not address the peer-to-peer coordination that a swarm needs.

3.2 Peer-to-Peer UAV Authentication

Peer authentication among UAVs has been studied with symmetric credentials [3] and in cross-domain settings [9, 12]. A recurring weakness is that peer trust is rooted in a single shared secret, so one captured node endangers the whole mesh. Our design instead roots peer trust in *per-pair* credentials, confining the blast radius of a capture.

3.3 Lightweight Hash Functions for Constrained Devices

SPONGENT [15] and related lightweight primitives [14] target minimal hardware area. We keep a hash-agnostic interface so that a device may run SHA3 in software or SPONGENT in hardware without changing the protocol.

3.4 Comparison of Existing Approaches

Table 1 contrasts our design with representative alternatives along the security features that matter for a swarm.

Table 1: Feature Comparison of UAV/IoT Authentication Approaches

Approach	PUF	Peer auth	No stored key	Fwd sec	Err corr	Formal proof
RSA-2048 PKI	No	Yes	No	Opt.	N/A	Partial
ECC-256	No	Yes	No	Opt.	N/A	Partial
PSK (AES-GCM)	No	Yes	No	No	N/A	Partial
Gope [2]	Yes	No	Yes	No	Simple	Informal
Li [9]	Yes	Partial	Yes	No	BCH	Informal
Khan [12]	Yes	No	Yes	Yes	BCH	Informal
Bera [3]	No	Yes	No	No	N/A	Informal
This work	Yes	Yes	Yes	Yes	BCH	Games + symbolic

Our protocol is the only entry that simultaneously provides a PUF root of trust, direct peer-to-peer authentication, no stored key on the device, mandatory forward secrecy, BCH-based error correction, and a security argument combining game-based reductions with a mechanised symbolic model.

4 Proposed Protocol Design

This section presents the four-phase protocol in full. We begin with the threat model and design goals, fix the notation, and then specify each phase message by message, with the mathematics of every step accompanied by a plain explanation of what it does and why it is secure.

4.1 Threat Model and Design Goals

4.1.1 Threat Model

We consider a Dolev–Yao adversary [22] augmented with physical capabilities: (1) **Network adversary**—may eavesdrop, intercept, modify, replay, delay, or drop any wireless message; (2) **Active challenger**—may pose as the GS toward an honest UAV and send adversarially chosen challenges and nonces; (3) **Physical capture**—may seize UAVs and read their NVM, but cannot clone PUF hardware, since invasive probing destroys it; (4) **Computational bounds**—a probabilistic polynomial-time (PPT) adversary cannot invert the hash, forge a MAC or authenticated ciphertext, break PUF unclonability, or solve the computational Diffie–Hellman problem, except with negligible probability; (5) **Side channels**—standard countermeasures (constant-time primitives, power-analysis resistance) are assumed.

4.1.2 Design Goals

The protocol targets nine properties: (1) mutual authentication in both directions, verified by keyed MACs; (2) anonymity via rotating temporary identities; (3) replay and man-in-the-middle resistance through nonces, timestamps, and transcript binding; (4) *no stored secret* on the UAV, since the master key is regenerated from the PUF on demand; (5) GS-free peer authentication rooted in per-pair credentials; (6) perfect forward secrecy through a mandatory ephemeral key exchange; (7) lightweight operation (only a hash, keyed MAC, AEAD, KDF, and one elliptic curve); (8) scalability, since the $O(N^2)$ peer authentications are independent and parallelizable; and (9) noise tolerance through the BCH fuzzy extractor.

4.2 System Model and Notation

The system consists of one GS with ample computation and tamper-resistant storage, and N UAVs, each carrying a PUF, a lightweight processor, a radio, and a small non-volatile memory. Table 2 lists the notation used throughout.

Table 2: Protocol Notation

Symbol	Description
UAV_i	The i -th Unmanned Aerial Vehicle
GS	Ground Station (enrollment and credential authority)
ID_i	Permanent identity of UAV_i (64-bit)
TID_i	Rotating temporary identity for anonymity
PUF_i	Physical Unclonable Function of UAV_i
$\mathbf{C}_i$	Enrollment challenge block $(C_i^{(1)}, \dots, C_i^{(L)})$
R, R'	Enrollment response; noisy field response
$\text{FE} = (\text{Gen}, \text{Rep})$	Fuzzy extractor (§2.4)
$P_i = (s_i, \text{seed}_i)$	<i>Public</i> helper data for UAV_i
mk_i	Stable per-device master key (never stored on the UAV)
mk_{GS}	GS-only master key for pairwise credentials
Cred_{ij}	Per-pair credential shared by $\text{UAV}_i, \text{UAV}_j$
K_{auth}^i	UAV–GS MAC key, $= \text{KDF}(\text{mk}_i; \text{p2-auth})$
K_{auth}^{ij}	Peer MAC key, $= \text{KDF}(\text{Cred}_{ij}; \text{peer-auth})$
$\text{SK}_i, \text{SK}_{ij}$	Session keys (Phase 2 / Phase 3)
$h(\cdot)$	160-bit hash: SHA3-256 ₁₆₀ or SPONGENT-160
$\text{MAC}_K(\cdot)$	Keyed message-authentication code
$\text{AE.Enc}, \text{AE.Dec}$	Authenticated encryption (AEAD)
$\text{KDF}(\cdot; \ell)$	Key-derivation function with label ℓ
g, g^a, g^{ab}	Generator; ephemeral public share; DH secret
N_x, n_x	Random nonces (128-bit)
T_x	Timestamp (32-bit)
$\oplus, \parallel$	Bitwise XOR; concatenation

Figure 1 shows how the four phases fit together: a one-time secure enrollment, an in-field UAV–GS handshake that also distributes peer credentials, GS-free peer handshakes, and the session keys that result.

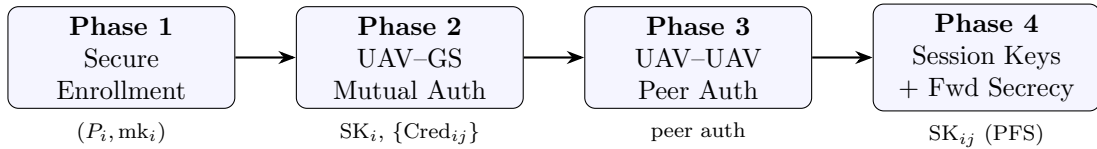


Figure 1: The four phases and what each produces. Enrollment is performed once, in a secure facility; Phases 2–4 run in the field.

4.3 Phase 1: Secure Enrollment and Registration

Enrollment runs once per UAV, in a physically secure facility over a trusted channel (a wired link or a shielded RF cage), before the drone is deployed. Its purpose is to establish the stable master key mk_i and the public helper data P_i , so that in the field the UAV can regenerate mk_i from its own PUF without ever storing a secret.

Step 1.1: PUF interrogation. The GS draws a block of L random 128-bit challenges $\mathbf{C}_i = (C_i^{(1)}, \dots, C_i^{(L)})$ and has the UAV evaluate its PUF on each:

$$R = (\text{PUF}_i(C_i^{(1)}), \dots, \text{PUF}_i(C_i^{(L)})). \quad (1)$$

Concatenating L blocks aggregates enough physical entropy to survive the helper-data loss quantified below; R is the raw enrollment response.

Step 1.2: Key and helper-data generation. The GS runs the fuzzy-extractor generator Gen on R . It samples a random seed, forms a code-offset sketch, and extracts the master key:

$$c = \text{C.Encode}(r), \quad r \xleftarrow{\$} \{0, 1\}^k, \quad s = R \oplus c, \quad (2)$$

$$\text{mk}_i = \text{KDF}(\text{Ext}_{\text{seed}}(R); \text{root}), \quad P_i = (s, \text{seed}). \quad (3)$$

Here s is the public sketch that will later let a noisy response be decoded back to R , and Ext is a strong extractor (realised by the KDF) that compresses R into a near-uniform key. The pair (P_i, mk_i) is the output of Gen .

Step 1.3: Sizing the enrollment. Because the sketch s leaks at most $n - k = 124$ bits about each block (Lemma 1), the number of blocks L is chosen so that the entropy left after the leak still exceeds the desired key length ℓ :

$$\sum_{b=1}^L (m_b - (n - k)) \geq \ell + 2 \log(1/\varepsilon), \quad (4)$$

where m_b is the measured per-block min-entropy and ε is a negligible statistical-distance target. This inequality is what guarantees mk_i is a strong key rather than a slightly-random one.

Step 1.4: Identity and pairwise-credential setup. The GS assigns a permanent identity and an initial rotating temporary identity, and fixes a GS-only master key from which all per-pair credentials are derived:

$$\text{TID}_i = h(\text{ID}_i \parallel \text{seed}'_i), \quad \text{Cred}_{ij} = \text{KDF}(\text{mk}_{\text{GS}}; \text{pair-cred} \parallel \min(i, j) \parallel \max(i, j)). \quad (5)$$

The temporary identity hides the permanent one from eavesdroppers, and each Cred_{ij} is a distinct secret for exactly one pair. These credentials are *not* released here; they are delivered, encrypted, during Phase 2.

Step 1.5: Storage. After enrollment:

- **GS stores** $\{\text{ID}_i, \text{TID}_i, \text{C}_i, P_i, \text{mk}_i\}$ per UAV, plus the single master key mk_{GS} , all in tamper-resistant memory.
- **UAV_i stores** $\{\text{TID}_i, \text{C}_i, P_i\}$ only. Every one of these is public; *no secret is stored on the drone*. This is the central advantage of the design: capturing a UAV yields no key material, because the master key exists only transiently, regenerated from the PUF whenever it is needed.

4.4 Phase 2: UAV–Ground Station Mutual Authentication

Phase 2 runs when a UAV powers up in the field. It is a four-message exchange that mutually authenticates the UAV and GS, establishes a forward-secret session key SK_i , and delivers the UAV's per-pair credentials for Phase 3. Both sides key their MACs with $K_{\text{auth}}^i = \text{KDF}(\text{mk}_i; \text{p2-auth})$: the GS computes it from the stored mk_i , and the UAV regenerates mk_i from its PUF. Figure 2 shows the message flow.

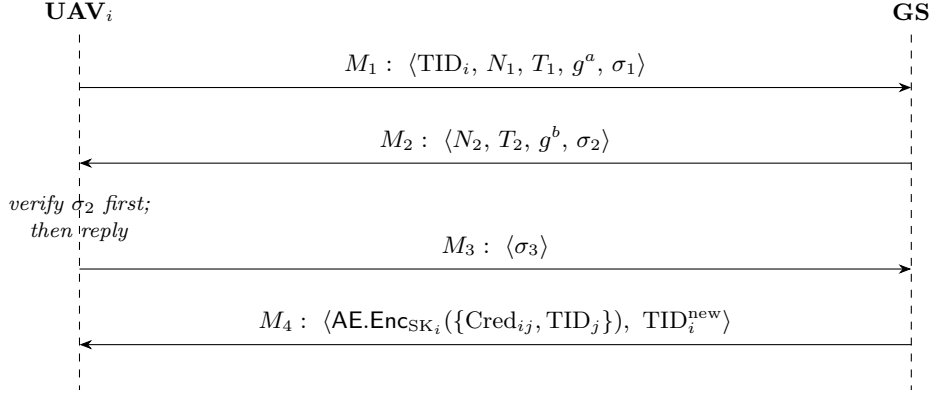


Figure 2: Phase 2 message sequence. The UAV validates the GS’s MAC before it replies, so a fraudulent challenger receives no response; peer credentials are delivered under authenticated encryption only after mutual authentication succeeds.

Message 1 (Authentication request): $\text{UAV}_i \rightarrow \text{GS}$. The UAV picks a fresh nonce and an ephemeral Diffie–Hellman scalar, and authenticates the opening message:

$$N_1 \xleftarrow{\$} \{0, 1\}^{128}, \quad a \xleftarrow{\$} \text{scalar}, \quad (6)$$

$$\sigma_1 = \text{MAC}_{K_{\text{auth}}^i}(\text{M1} \parallel \text{TID}_i \parallel N_1 \parallel T_1 \parallel g^a), \quad (7)$$

$$M_1 = \langle \text{TID}_i, N_1, T_1, g^a, \sigma_1 \rangle. \quad (8)$$

Because σ_1 is a MAC keyed by K_{auth}^i , only a party that holds mk_i —that is, the genuine UAV (through its PUF) or the GS (through its database)—can produce it. The tag M1 marks the message type so that a message from one step can never be replayed as another.

Message 2 (GS response): $\text{GS} \rightarrow \text{UAV}_i$. The GS checks the timestamp and the nonce for freshness, looks up mk_i , verifies σ_1 , and answers with its own nonce and ephemeral share:

$$N_2 \xleftarrow{\$} \{0, 1\}^{128}, \quad b \xleftarrow{\$} \text{scalar}, \quad (9)$$

$$\sigma_2 = \text{MAC}_{K_{\text{auth}}^i}(\text{M2} \parallel \text{TID}_i \parallel N_1 \parallel N_2 \parallel T_2 \parallel g^a \parallel g^b), \quad (10)$$

$$M_2 = \langle N_2, T_2, g^b, \sigma_2 \rangle. \quad (11)$$

The tag σ_2 authenticates the GS to the UAV: it binds both nonces and both ephemeral shares under the shared key, so no network adversary can forge or alter it.

Message 3 (UAV confirmation): $\text{UAV}_i \rightarrow \text{GS}$. The UAV first regenerates its master key from its PUF and *verifies σ_2 before doing anything else*:

$$\text{mk}_i = \text{Rep}(\text{PUF}_i(C_i), P_i), \quad (12)$$

$$\text{abort silently if } \sigma_2 \text{ is invalid.} \quad (13)$$

This “verify-before-respond” rule is what prevents a fraudulent GS from using the UAV as an oracle: an attacker who cannot forge σ_2 elicits no reply at all. If the check passes, the UAV confirms the transcript:

$$\sigma_3 = \text{MAC}_{K_{\text{auth}}^i}(\text{M3} \parallel \mathcal{T}), \quad M_3 = \langle \sigma_3 \rangle, \quad (14)$$

where the transcript $\mathcal{T}$ is the concatenation of all fields of M_1 and M_2 . Note that the raw PUF response is consumed inside Rep and *never appears on the wire*; only a MAC over public values is

sent.

Message 4 (Confirmation and credential delivery): $\text{GS} \rightarrow \text{UAV}_i$. After verifying σ_3 , both parties hold the same ingredients, so each computes the forward-secret session key

$$\text{SK}_i = \text{KDF}(\text{mk}_i \parallel g^{ab} \parallel \mathcal{T}; \text{p2-sess}), \quad (15)$$

and the GS delivers the UAV's per-pair credentials and a fresh identity under authenticated encryption keyed by SK_i :

$$\text{CredPkg} = \text{AE.Enc}_{\text{SK}_i}(\eta; \text{TID}_i; \{(\text{Cred}_{ij}, \text{TID}_j)\}_{j \neq i}), \quad (16)$$

$$\text{TID}_i^{\text{new}} = h(\text{TID}_i \parallel \text{mk}_i \parallel N_1 \parallel N_2), \quad M_4 = \langle \text{CredPkg}, \text{TID}_i^{\text{new}} \rangle. \quad (17)$$

Authenticated encryption gives the credentials both confidentiality and integrity: an eavesdropper learns nothing about any Cred_{ij} , and a tampered package is rejected. The session key mixes in the Diffie–Hellman secret g^{ab} , which is what makes it forward-secret (Phase 4). After Phase 2 the UAV holds SK_i , its peer credentials $\{\text{Cred}_{ij}\}$, and a rotated identity.

4.5 Phase 3: UAV-to-UAV Peer Authentication

Phase 3 lets any two UAVs authenticate each other directly, using the per-pair credential Cred_{ij} that each received (encrypted) in Phase 2. It needs no GS involvement, so it works even when the GS link is down. Both sides key their MACs with $K_{\text{auth}}^{ij} = \text{KDF}(\text{Cred}_{ij}; \text{peer-auth})$. Figure 3 shows the three-message flow.

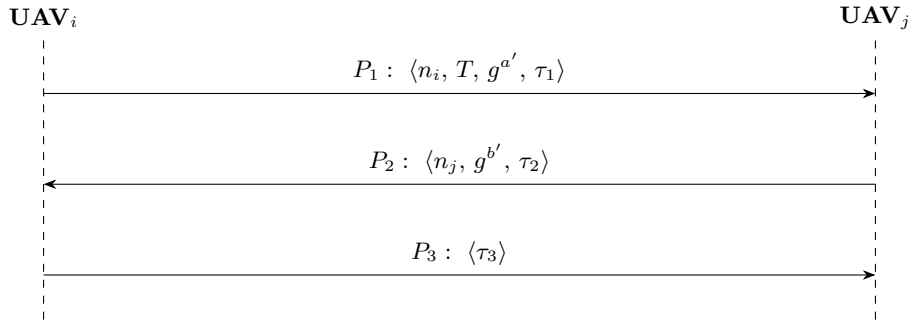


Figure 3: Phase 3 message sequence. Every token is a MAC keyed by the per-pair credential Cred_{ij} and tagged with its message type, so tokens cannot be reflected or reused across steps.

Message P1 (Peer request): $\text{UAV}_i \rightarrow \text{UAV}_j$. The initiator picks a fresh nonce and ephemeral share and sends an authenticated request:

$$n_i \xleftarrow{\$} \{0, 1\}^{128}, \quad a' \xleftarrow{\$} \text{scalar}, \quad (18)$$

$$\tau_1 = \text{MAC}_{K_{\text{auth}}^{ij}}(\text{P1} \parallel i \parallel j \parallel n_i \parallel T \parallel g^{a'}), \quad P_1 = \langle n_i, T, g^{a'}, \tau_1 \rangle. \quad (19)$$

Because τ_1 is keyed by Cred_{ij} , which only UAV_i and UAV_j possess, a third party—even one holding some other pair's credential—cannot produce it.

Message P2 (Peer response): $\text{UAV}_j \rightarrow \text{UAV}_i$. The responder verifies τ_1 , then answers with its own nonce and share:

$$\tau_2 = \text{MAC}_{K_{\text{auth}}^{ij}}(\text{P2} \parallel i \parallel j \parallel n_i \parallel n_j \parallel g^{a'} \parallel g^{b'}), \quad P_2 = \langle n_j, g^{b'}, \tau_2 \rangle. \quad (20)$$

τ_2 binds both identities and both nonces, so it authenticates UAV_j to UAV_i and confirms that both are in the same fresh session.

Message P3 (Completion): $\text{UAV}_i \rightarrow \text{UAV}_j$. The initiator verifies τ_2 and closes the hand-

shake with a final tag over the transcript $\mathcal{T} = P_1 \| P_2$:

$$\tau_3 = \text{MAC}_{K_{\text{auth}}^{ij}}(P_3 \| \mathcal{T}), \quad P_3 = \langle \tau_3 \rangle. \quad (21)$$

The three distinct type tags (P_1, P_2, P_3) ensure a token from one step can never be replayed as another, which rules out reflection attacks. For a swarm of N UAVs, full-mesh peer authentication runs $N(N-1)/2$ independent Phase 3 exchanges (for example, 45 for $N = 10$), all of which may proceed in parallel.

4.6 Phase 4: Session Key Establishment and Forward Secrecy

Phase 4 is the keying that both interactive phases finish with. In Phase 2 the UAV–GS key is SK_i of Eq. (15); in Phase 3 the peer key is

$$\text{SK}_{ij} = \text{KDF}(\text{Cred}_{ij} \| g^{a'b'} \| \mathcal{T}; \text{peer-sess}). \quad (22)$$

Both keys have the same shape, which we write uniformly as

$$\text{SK} = \text{KDF}\left(\underbrace{k_{\text{lt}}}_{\text{mk}_i \text{ or } \text{Cred}_{ij}} \| g^{xy} \| \mathcal{T}; \ell\right). \quad (23)$$

Two independent secrets go into every session key. The first is the long-term key k_{lt} , which ties the session to the PUF (via mk_i) or to the pair (via Cred_{ij}). The second is the ephemeral Diffie–Hellman secret g^{xy} , whose public shares were authenticated inside the MAC transcript of the corresponding handshake. Mixing in g^{xy} is what provides *perfect forward secrecy*: once the ephemeral scalars are erased, an adversary who later learns k_{lt} still cannot reconstruct a past key, because doing so would require solving Diffie–Hellman on the recorded shares. The exchange is mandatory—there is no derivation path that omits g^{xy} —so a session either establishes a forward-secret key or fails.

5 Security Analysis

We now analyse the protocol formally. Each theorem is proved by reducing an attack to the violation of a standard assumption; the reductions are then mirrored by a symbolic model (§5.8).

5.1 Assumptions

The proofs rely on the following standard assumptions.

1. **Hash security.** h is pre-image, second-pre-image, and collision resistant; in the reductions it is modelled as a random oracle.
2. **MAC unforgeability.** MAC is existentially unforgeable under chosen-message attack (EUF-CMA): without the key, a fresh valid tag can be produced only with negligible probability.
3. **AEAD security.** AE provides ciphertext indistinguishability (IND-CCA) and ciphertext integrity (INT-CTXT).
4. **PUF unpredictability.** Given polynomially many challenge–response pairs, the response to a fresh challenge is computationally indistinguishable from uniform to an adversary that does not possess the device; invasive probing destroys the PUF.
5. **Diffie–Hellman.** The gap computational Diffie–Hellman (gap-CDH) problem is hard in $\mathbb{G}$.

We first record the helper-data property that underpins the strength of the master key.

Lemma 1 (Helper-data privacy). *Let the enrollment response R have conditional min-entropy $\tilde{H}_\infty(R) = m$. The code-offset sketch $s = R \oplus c$ discloses at most $n - k$ bits, so*

$$\tilde{H}_\infty(R | P) \geq m - (n - k). \quad (24)$$

Consequently, if the extractor output length ℓ satisfies $\ell \leq m - (n - k) - 2 \log(1/\varepsilon)$, then mk is within statistical distance ε of uniform given the public helper data P .

Proof. Given the internal randomness r , the sketch $s = R \oplus c$ is a deterministic function of R that pins R to a coset of the $[n, k]$ code $\mathbf{C}$; revealing that coset fixes at most $n - k$ syndrome bits, so by the chain rule for average min-entropy $\tilde{H}_\infty(R | s) \geq \tilde{H}_\infty(R) - (n - k)$, and the seed is independent of R . Applying the strong-extractor property to the residual source gives a key within distance ε of uniform. $\square$ $\square$

5.2 Phase 2: Mutual Authentication

Theorem 2 (Phase 2 mutual authentication). *After a successful Phase 2 exchange, UAV_i and the GS have mutually authenticated one another, and both agree on the transcript $\mathcal{T}$, except with negligible probability.*

Proof. Both directions rest on the MAC key $K_{\text{auth}}^i = \text{KDF}(\text{mk}_i; \text{p2-auth})$, which by Lemma 1 and PUF unpredictability is unknown to any adversary lacking either the PUF or the GS database. (*GS authenticates UAV.*) The GS accepts only if σ_1 (and later σ_3) verifies under K_{auth}^i ; producing a fresh valid tag without the key is exactly an EUF-CMA forgery. (*UAV authenticates GS.*) Symmetrically, the UAV accepts only if σ_2 verifies; although every field inside σ_2 is public, the *key* is secret, so a fresh valid σ_2 is again a forgery. A nonce collision that could enable transcript reuse occurs with probability at most $q^2/2^{128}$ for q sessions. Absent forgery and collision, the two accepting parties share identical fields in M_1 and M_2 , i.e. they agree on $\mathcal{T}$. $\square$ $\square$

Corollary 3 (No PUF read-out oracle). *An adversary that impersonates the GS learns no PUF response, so it cannot harvest challenge–response pairs to model the PUF.*

Proof. By the verify-before-respond rule, the UAV emits M_3 only after σ_2 verifies, which by Theorem 2 an adversary can force only by forging a MAC. When the check fails the UAV sends nothing; when it passes, the only value on the wire is σ_3 , a MAC over public transcript fields—the PUF response is consumed inside Rep and is never transmitted. Hence no challenge–response pair is exposed. $\square$ $\square$

5.3 Replay Attack Resistance

Theorem 4 (Replay resistance). *Phase 2 and Phase 3 resist replay attacks.*

Proof. Three mechanisms combine. First, every message carries a fresh 128-bit nonce and a timestamp verified within a window ΔT ; a verbatim replay reuses a stale nonce or timestamp and is rejected. Second, each MAC binds a per-message type tag ($\text{M1}, \dots, \text{P3}$), so a token from one step cannot be accepted at another. Third, from the second message onward each MAC binds the partner’s nonce, so a token from one session cannot be reused in another without producing a mismatch—which, by Theorems 2 and 5, would require a forgery. $\square$ $\square$

5.4 Phase 3: Peer Mutual Authentication

Theorem 5 (Peer mutual authentication). *After a successful Phase 3 exchange, UAV_i and UAV_j have mutually authenticated one another, without GS involvement, and no per-pair secret is exposed in transit.*

Proof. The credential Cred_{ij} reaches each endpoint only inside the Phase 2 package $\text{AE.Enc}_{\text{SK}_i}(\dots)$; by INT-CTXT the adversary cannot make a UAV accept a substituted credential, and by IND-CCA it learns nothing about Cred_{ij} . Given a secret Cred_{ij} , the tokens τ_1, τ_2, τ_3 are MACs under K_{auth}^{ij} ; each binds both identities and both nonces, so a fresh acceptance forces a matching computation by the partner—otherwise an EUF-CMA forgery. Because credentials are pair-specific, a third party holding some other $\text{Cred}_{k\ell}$ has no information about Cred_{ij} and cannot assert identity i to j . $\square$ $\square$

5.5 Man-in-the-Middle Resistance

Theorem 6 (MITM resistance). *The protocol resists active man-in-the-middle attacks in both Phase 2 and Phase 3.*

Proof. Every accepted message is a MAC (or an authenticated ciphertext) over a transcript that includes fresh nonces, a timestamp, a type tag, and—from the second message onward—the partner’s nonce and ephemeral share. Altering any field invalidates the covering tag, and forging a new tag requires the secret key (EUF-CMA / INT-CTXT). By the agreement guarantees of Theorems 2 and 5, both endpoints therefore hold the identical transcript, so an interposed adversary cannot sit undetected between them. $\square$ $\square$

5.6 Session-Key Secrecy and Forward Secrecy

Theorem 7 (Session-key secrecy). *For an adversary that corrupts neither endpoint of the target session, the session key of Eq. (23) is indistinguishable from uniform.*

Proof. The key is $\text{KDF}(k_{\text{lt}} \| g^{xy} \| \mathcal{T})$. Two independent secrets protect it: the long-term key k_{lt} (mk_i by Lemma 1 and PUF unpredictability, or Cred_{ij} by IND-CCA of its delivery, Theorem 5) and the Diffie–Hellman secret g^{xy} . Modelling the KDF as a pseudorandom function, replacing it by a random function is undetectable up to its PRF advantage; the input g^{xy} is unpredictable under gap-CDH given only the authenticated shares g^x, g^y . Hence the output is uniform up to negligible terms. $\square$ $\square$

Theorem 8 (Perfect forward secrecy). *If the ephemeral scalars are erased after use, then revealing the long-term secrets (mk_i , Cred_{ij} , or mk_{GS}) after a target session closes does not compromise that session’s key.*

Proof. A post-session reveal gives the adversary k_{lt} but not the erased scalars x, y . The key of Eq. (23) still depends on g^{xy} , which the adversary must now compute from the recorded, authenticated shares g^x, g^y —a gap-CDH instance. Because the ephemeral exchange is mandatory, there is no non-ephemeral key to fall back on. Hence past keys remain indistinguishable from uniform. $\square$ $\square$

5.7 Physical Capture Resistance

Theorem 9 (Physical capture resistance). *Capturing UAV_k and reading its NVM does not let the adversary impersonate any other UAV to the GS, nor impersonate any peer pair that does not involve k . A captured node’s peer exposure is limited to its own $N - 1$ links.*

Proof. The NVM of UAV_k holds only $\{\text{TID}_k, \mathbf{C}_k, P_k, \{\text{Cred}_{kj}\}_j\}$ plus any live session keys; all of $\text{TID}_k, \mathbf{C}_k, P_k$ are public. Impersonating UAV_m ($m \neq k$) in a fresh Phase 2 needs $K_{\text{auth}}^m = \text{KDF}(\text{mk}_m; \cdot)$; but mk_m is never stored, is not recoverable from k ’s silicon (tamper evidence), and is unpredictable (PUF assumption), so this reduces to a MAC forgery. Impersonating a pair (m, ℓ) with $k \notin \{m, \ell\}$ needs $\text{Cred}_{m\ell} = \text{KDF}(\text{mk}_{\text{GS}}; \cdot)$, which is independent of every captured Cred_{kj} (distinct KDF inputs, and mk_{GS} never leaves the GS). Only pairs that include k share a captured credential, giving the stated $N - 1$ exposure. $\square$ $\square$

5.8 Symbolic Verification

The computational theorems are corroborated by a symbolic model in the Tamarin prover [28], whose multiset-rewriting calculus expresses freshness, stateful credential issuance, and device-compromise events natively. In the model, $h, \text{MAC}, \text{AE}, \text{KDF}$ are function symbols with the usual equational theory, each PUF is a private per-device function, and helper data and all wire terms are public. The Dolev–Yao adversary controls the network; two extra rules model compromise—one that reveals a captured node’s NVM (its identities, helper data, and per-pair credentials, but neither its PUF nor its master key) and one that reveals long-term secrets after a session closes, for the forward-secrecy query. The prover discharges lemmas that correspond one-to-one with the theorems above:

injective agreement for Phase 2 and Phase 3 (Theorems 2 and 5), a no-replay lemma (Theorem 4), a capture-isolation lemma (Theorem 9), a session-key-secrecy lemma (Theorem 7), and a forward-secrecy lemma (Theorem 8). The PUF’s noise is idealised as a noiseless private function—its error tolerance is the computational correctness property of the fuzzy extractor—and confidential delivery is abstracted by a symmetric encryption operator, leaving the fine-grained secrecy argument to the computational analysis.

5.9 Summary of Security Properties

Table 3 summarises the guarantees and the mechanism behind each.

Table 3: Summary of Security Properties

Property	Result	Mechanism
UAV–GS mutual auth	Thm 2	Keyed MAC under PUF-derived mk_i
No PUF read-out oracle	Cor 3	Verify-before-respond; response stays on-device
Replay resistance	Thm 4	Nonces + timestamps + type tags
Peer mutual auth	Thm 5	Per-pair credential + keyed MAC
MITM resistance	Thm 6	Transcript-binding MACs / AEAD
Session-key secrecy	Thm 7	KDF over long-term key and DH secret
Perfect forward secrecy	Thm 8	Mandatory authenticated ephemeral DH
Physical-capture resistance	Thm 9	No stored key; per-pair credentials
Identity anonymity	—	Rotating temporary identities

6 Reference Implementation

The protocol of Section 4 is implemented in C++17 on OMNeT++ 6.3. This section records the concrete primitive choices, the parameterisation of the fuzzy extractor, and the verification evidence for each component, so that every number in Section 7 can be traced to a specific mechanism.

6.1 Structure

The implementation separates protocol logic from simulation machinery. The classes implementing Phases 1–4 depend only on abstract cryptographic interfaces and on byte strings; they contain no reference to the simulation kernel. This allows the entire four-phase exchange to be driven over an in-memory transport and validated independently of the network model, which is how the protocol-level results in Section 7.2 were obtained.

A single transmission module owns the link-delay model, so propagation, serialisation, processing delay, and jitter are defined once rather than duplicated per node type. The same module provides the interception point used by the adversary of Section 7.9; when no adversary is present it forwards immediately, so attack and baseline runs remain directly comparable.

6.2 Canonical message encoding

Every message is serialised by one encoder, whose output serves simultaneously as the wire image, the input to the message-authentication code, and the byte count driving the link delay. Each field is emitted as a type tag, a two-byte length, and its value, under a header carrying the protocol version, the suite identifier, and a per-message-type tag:

$$\text{field}(t, v) = \text{u8}(t) \parallel \text{u16}(|v|) \parallel v, \quad (25)$$

$$\text{enc} = \text{u8}(\text{ver}) \parallel \text{u8}(\text{suite}) \parallel \text{u8}(\text{type}) \parallel \text{u16}(k) \parallel \text{field}(t_1, v_1) \cdots \quad (26)$$

with tags emitted in strictly ascending order. The encoding is therefore *injective* (no two distinct field sets share a byte string) and *canonical* (each field set has exactly one valid encoding), which is the precondition the unforgeability argument of Section 5 requires of its MAC inputs. Because the same encoder produces the transmitted bytes, reported communication overhead cannot drift from what is actually sent. Non-canonical input — duplicate or out-of-order tags — is rejected rather than accepted, and reading an absent field raises an error instead of yielding an empty buffer.

6.3 Cryptographic suites

Two interoperable suites implement the same abstract interface, selected by one parameter. Table 4 lists them.

Table 4: The two cryptographic profiles

Component	Profile sha3	Profile sponge
Hash $h(\cdot)$	SHA3-256 truncated to 160 bits	SPONGENT-160/160/16
MAC	HMAC-SHA3-256, 128-bit tag	prefix keyed sponge, 128-bit tag
KDF	HKDF (extract-then-expand)	one-pass sponge
AEAD	ChaCha20-Poly1305	Ascon-128a v1.2
Key exchange	X25519	X25519
Hash/MAC level	128-bit	80-bit ($c = 160$)
Provenance	OpenSSL 3.5	vendored

Three points require explicit statement.

HMAC is not applicable to the sponge profile. HMAC is defined over a compression function of block size B and requires $|K| \leq B$. For SPONGENT-160/160/16 the rate is 16 bits, so the block is two bytes — smaller than any useful key — and hashing the key first still exceeds it. The construction is therefore undefined, and any ad-hoc substitution would have no supporting proof. The sponge profile instead uses the standard prefix keyed sponge underlying KMAC [20], which is sound for a sponge and does not require HMAC’s nested structure: a sponge with non-zero capacity is not length-extendable, so the nesting HMAC exists to provide is unnecessary.

The sponge profile operates at the 80-bit level. With capacity $c = 160$ the keyed-sponge bound gives approximately $c/2 = 80$ bits, matching the collision resistance of a 160-bit digest but deliberately asymmetric with the 128-bit AEAD and key-exchange components. It is presented as a constrained-hardware profile at that level rather than as a 128-bit one.

Key derivation is domain-separated. Distinct leading bytes separate the hash, MAC, and KDF modes even where they share a permutation, and every derived key carries a unique label (p2-auth, p2-sess, pair-cred, peer-auth, peer-sess, tid-rotate), so a key minted for one purpose cannot be mistaken for another.

All protocol randomness — nonces, ephemeral scalars, extractor seeds, enrollment challenges — is drawn from a deterministic generator seeded from the simulator’s random-number stream. Ephemeral X25519 keys are loaded from those bytes rather than generated internally by the library, so an entire run, including every ephemeral key, is reproducible from its configured seed.

6.4 Error-correcting code

The generator polynomial is formed as the product of the minimal polynomials of $\alpha^1, \dots, \alpha^{2^t}$ over the *complete* conjugate closure of each root. For $t = 18$ over $\text{GF}(2^8)$ the closure contains 124 roots, giving $\deg g = 124$ with binary coefficients and therefore

$$\text{BCH}(n=255, k=131, t=18), \quad n - k = 124 \text{ parity bits.} \quad (27)$$

Both properties are asserted at construction: a non-binary coefficient indicates an incomplete closure, and the parity length is computed rather than assumed. This matters because $k = 131 \geq 128$ is

what allows a full response block to be protected; taking the parity length as $mt = 144$ instead would give $k = 111$ and silently leave the tail of every response uncovered.

Decoding proceeds by syndrome computation, the Berlekamp–Massey algorithm, and a Chien search, followed by a verification that the corrected word has all-zero syndromes. Failure is reported as failure: the decoder never returns the uncorrected input while indicating success.

6.5 Fuzzy-extractor parameterisation

Enrollment applies **Gen** to a response of L blocks of $n = 255$ bits; reproduction applies **Rep** with the public helper data. Two consequences of Lemma 1 drive the parameter choice.

First, since the sketch discloses up to $n - k$ bits per block, the block length must exceed the leak by a useful margin. A 128-bit block against a 124-bit leak contributes almost nothing, which is why blocks are 255 bits — matching the code length. A construction that sized blocks to the native response width of a 128-bit challenge–response pair could not satisfy Eq. (28) at any block count. Second, the budget

$$L(\rho n - (n - k)) \geq \ell + 2 \log_2(1/\varepsilon) \quad (28)$$

is enforced at construction and raises an error when unmet, rather than being documented and left to the caller. With $\rho = 0.95$, $\ell = 256$ and $\varepsilon = 2^{-64}$, each block contributes 118.25 bits and $L = 4$ gives $473 \geq 384$.

Three profiles are provided (Table 5). The default uses the parameters of Section 4; the third concatenates a $[3, 1, 3]$ repetition inner code, which must operate on repeated measurements of the same physical bit. Enrollment fixes a reference and publishes each copy’s offset as additional helper data; reproduction removes those offsets before voting, reducing the per-bit error rate from p to approximately $3p^2$. The inner offsets are charged to the leak accounting at $(\text{rep} - 1)$ bits per outer bit.

Table 5: Fuzzy-extractor profiles

Profile	Code	L	PUF bits	Leak (bits)
bch255-131-18 (default)	BCH(255,131,18)	4	1020	496
bch255-91-25	BCH(255,91,25)	5	1275	820
rep3-bch255-131-18	$[3, 1, 3] \oplus \text{BCH}(255,131,18)$	5	3825	3170

Reproduction fails closed. If any block fails to decode, no key material is returned: a partially recovered response is never converted into a key.

6.6 PUF models

Three models are provided, and results state which one they rest on. An *idealised* model realises the strong-PUF abstraction as a keyed pseudorandom function and is used where near-uniform responses are the intended assumption. An *arbiter* model implements the additive-delay formulation over 128 stages with per-device Gaussian delay weights. A third XORs k arbiter chains.

In the arbiter model, measurement noise is added to the accumulated *delay difference* rather than by flipping output bits:

$$\Delta' = \sum_i w_i \phi_i + \mathcal{N}(0, \sigma), \quad r = \mathbb{K}[\Delta' > 0]. \quad (29)$$

Bits whose delay difference lies near zero therefore flip far more often than the rest, reproducing the reliability profile of real silicon. The measured variance of per-bit flip rates is approximately $98\times$ that of a uniform bit-flip channel, a structural property a per-bit model cannot exhibit. A calibration routine converts a target average bit-error rate into the corresponding σ .

Measured characteristics over 24 devices and 276 device pairs: inter-device Hamming distance 0.5019 for the arbiter model and 0.5026 for the idealised model, both consistent with the ideal 0.5;

calibrated bit-error rates track their targets to within 0.2 percentage points across 1%–10%.

6.7 Verification of primitives

Each primitive is checked against published reference values where they exist, and reports its verification status programmatically. Table 6 summarises.

Table 6: Verification evidence per primitive

Primitive	Evidence
SHA3-256	FIPS 202 / CAVP short-message vectors
HMAC-SHA3-256	NIST example values with intermediate results
HKDF	RFC 5869, Appendices A.1–A.3
ChaCha20-Poly1305	RFC 8439, Section 2.8.2
X25519	RFC 7748, Section 6.1; low-order points rejected
Ascon-128a	all 1089 vectors of the official LWC known-answer-test file
SPONGENT-160	byte-exact against the designers' reference implementation and the published "Sponge + Present = Spongent" digest; no official known-answer-test file exists for SPONGENT, and none is claimed
BCH(255,131,18)	$\deg g = 124$; $g(x) \mid x^{255} + 1$; exact recovery in all 18,000 trials across weights 1–18; failure correctly reported beyond the radius in 3,000 trials

Neither vendored primitive has undergone timing or side-channel analysis; tag comparisons use a constant-time routine and no branch depends on secret data, but nothing stronger is claimed. The verification-status strings state this, and a regression test fails if the qualification is removed.

The complete unit suite comprises nine test programs and approximately 4.3×10^5 assertions, covering the primitives above, the encoding, the fuzzy extractor, the PUF models, and the full four-phase exchange.

6.8 Simulation model

The scenario places N UAVs around a 500×300 m field with the ground station at its centre. Link delay is the sum of propagation (from node coordinates), serialisation at the configured bit rate, a fixed processing delay, and half-normal jitter, each reported separately per message. This primary model does not itself simulate MAC-layer contention, so its reported network delay is a lower bound; §7.11 reports a second, independent measurement of the same Phase-2 handshake over a real IEEE 802.11 stack with genuine CSMA/CA contention, so that lower bound is now bracketed by an upper one rather than left undisclosed.

All node parameters are declared with defaults and are therefore overridable from the configuration file, and statistics are declared so that both scalar and vector outputs are recorded.

6.9 Evaluation scenarios

Every scenario in Section 7 changes exactly one variable relative to the reference configuration StadiumSHA3 and holds everything else fixed, so that a difference in outcome can be attributed to that one variable rather than to a confound. Table 7 lists all seventeen configurations, the single variable each isolates, and which table or figure reports its result, so the purpose of every run in the evaluation is stated in one place rather than left implicit.

Table 7: Every evaluation scenario, the variable it isolates, and where its result is reported

Scenario	Variable isolated	Question it answers	Reported in
StadiumSHA3	reference point: $N=10$, software-oriented suite	What does the protocol cost under its default, intended deployment?	Tab. 8–9
StadiumSPONGENT	crypto suite (<code>sha3</code> → <code>spongent</code>), N unchanged	Does swapping to the hardware-oriented suite change correctness, and by how much does it change cost?	Tab. 8–9
Baseline5UAV	swarm size, $N=5$	Lower end of the scaling curve	Tab. 8–9
Swarm20	swarm size, $N=20$	Upper end of the scaling curve; does per-pair Phase 3 cost stay flat as pair count grows to 190?	Tab. 8–9
ArbiterPuf	PUF model, idealised → physically-realistic additive-delay model, 3% native BER	Does the protocol still function against a PUF model with real timing-race noise rather than an idealised stand-in?	Tab. 8
HighNoise	PUF bit-error rate forced to 5%	Where does reliability start to degrade, and how is that degradation reported honestly (device never authenticates, rather than authenticates incorrectly)?	Tab. 8
NoiseSweep	BER swept 1–10% across three fuzzy-extractor profiles	Does the measured key-reproduction failure rate track the analytic binomial prediction, i.e. is the BCH decoder genuinely correcting errors?	Tab. 14, Fig. 5
MobilityLinear	UAV motion: constant velocity, boundary bounce	Does physical movement disturb the protocol's timing or correctness?	Tab. 16
MobilityRandomWalk	UAV motion: random heading redrawn every 100 ms	Cross-check of the mobility finding under a second, statistically different motion pattern	Tab. 16
Inet80211SHA3	network layer: idealised delay → real IEEE 802.11 CSMA/CA, <code>sha3</code> suite	What does genuine MAC-layer contention add to the idealised lower bound?	Tab. 17
Inet80211SPONGENT	network layer as above, <code>spongent</code> suite	Confirms the contention cost is suite-independent	Tab. 17
BaselineRSA	authentication mechanism: PUF-derived MAC → RSA-2048-signed ephemeral DH, same simulator	Fair, full-protocol comparison against a real PKI competitor	Tab. 13
BaselineECDSA	authentication mechanism: PUF-derived MAC → ECDSA-P256-signed ephemeral DH, same simulator	Fair, full-protocol comparison against a real ECC competitor	Tab. 13
AtkGsImpersonate	adversary present: forged ground-station challenge	Does a fraudulent GS elicit a reply from an honest UAV?	Tab. 15
AtkLegacyGsImpersonate	same attacker, verify-before-respond (P3) disabled	Negative control: does the same attacker succeed once the one relevant defence is removed?	Tab. 15
AtkReplayM1	adversary present: captured M_1 replayed inside the freshness window	Does the nonce cache reject a verbatim replay that the timestamp window alone would admit?	Tab. 15
AtkCredentialSniff	adversary present: passive read of M_4 's credential package	Does authenticated encryption keep the per-pair credential confidential in transit?	Tab. 15

Grouped by purpose, the seventeen scenarios fall into five families: a *reference and suite pair* (StadiumSHA3, StadiumSPONGENT), a *scaling triple* together with the reference (Baseline5UAV, StadiumSHA3, Swarm20, at $N=5, 10, 20$), a *reliability family* (ArbiterPuf, HighNoise, NoiseSweep) that separately stresses PUF realism and PUF noise, a *deployment-realism pair* (mobility and INET) that each bracket one simplification of the idealised model, and a *security family* of four attack configurations built around one negative control. No scenario duplicates another's variable: the two

mobility models and the two INET suites are kept deliberately minimal precisely because §7.10 and §7.11 show their outcomes agree with each other to within confidence interval, which is itself part of the finding, not an omission.

7 Evaluation

7.1 Methodology

Each configuration is executed for thirty independent repetitions with distinct seed sets (five for the attack configurations, individually justified in §7.9). The unit of replication is the *run*, not the individual record: observations within one run share a pseudorandom stream and a single scheduling epoch, so they are correlated. Confidence intervals are therefore computed in two stages — a per-run mean, then a *t*-interval across run means with $R - 1$ degrees of freedom. Pooling all records instead would inflate the degrees of freedom from $R - 1$ to $NR - 1$ and understate every interval by roughly $\sqrt{N}$. Intervals are reported as mean $\pm$ half-width at 95%; the analysis refuses to form an interval from fewer than three runs.

Proportions are accompanied by Wilson score intervals, which behave correctly near 0 and 1. Where no failure is observed, the rule-of-three upper bound is given: 100 successes in 100 trials bounds the failure probability at approximately 3%, and is reported as such rather than as perfect reliability.

Compute times are host wall-clock measurements; network delays are simulated. These are distinct clocks and are reported separately, never summed without qualification. Per-primitive costs are measured over batches of at least 10^3 iterations and reported as medians, because a single clock read (tens of nanoseconds) is not negligible against a microsecond-scale operation.

7.2 Functional correctness

Table 8 reports authentication outcomes. Two success rates are distinguished, because they answer different questions. The first counts UAVs that transmitted a first message; the second counts *all* enrolled devices. They diverge when a device cannot reproduce its master key, in which case it never initiates the exchange at all — and reporting only the former would conceal that failure mode entirely.

Table 8: Authentication outcomes, thirty repetitions per configuration

Configuration	Devices	Authenticated	Rate	Wilson 95%	Peer pairs
StadiumSHA3 ($N=10$)	300	300	1.000	[0.987, 1.000]	2700/2700
StadiumSPONGENT ($N=10$)	300	300	1.000	[0.987, 1.000]	2700/2700
Baseline5UAV ($N=5$)	150	150	1.000	[0.975, 1.000]	600/600
Swarm20 ($N=20$)	600	600	1.000	[0.994, 1.000]	11400/11400
ArbiterPuf (3% BER)	300	298	0.993	[0.976, 0.998]	2664/2690
HighNoise (5% BER)	300	247	0.823	[0.776, 0.862]	1830/2214

With the idealised PUF model every device authenticates in every configuration, including the 20-UAV swarm where all 11,400 peer-pair views complete. Under the arbiter model at 3% bit-error rate two devices in three hundred fail to regenerate their key; at 5%, fifty-three do. Those failures are reliability limits of the fuzzy extractor, not authentication failures, and are reported separately below.

7.3 Latency and communication cost

Table 9 reports the Phase-2 handshake’s simulated elapsed time, together with its compute and network components. These three columns are *not* one figure split into two: Compute and Network are independent measurements on two different clocks (host wall-clock timers for Compute, the simulator’s own clock for Network), and adding them gives the total a real device would pay if it ran the two costs back to back — a figure this paper uses explicitly in §7.7, never silently. Simulated

elapsed is the third, separate quantity: the OMNeT++ simulation clock’s own start-to-finish reading for the handshake, which does not advance while host compute time is being measured, so it is close to the Network column and systematically smaller than Compute+Network.

Table 9: Phase-2 latency and overhead (mean $\pm$ 95% CI, thirty runs). *Simulated elapsed* is the simulator-clock reading for the whole handshake, not Compute+Network — see §7.3 for why the three columns do not sum

Configuration	Simulated elapsed (ms)	Compute (ms)	Network (ms)	Overhead (B)
StadiumSHA3	1.5028 ± 0.0023	1.6024 ± 0.0322	1.0737 ± 0.0020	777
StadiumSPONGENT	1.5082 ± 0.0023	4.4533 ± 0.0306	1.0790 ± 0.0020	781
Baseline5UAV	1.1442 ± 0.0039	1.5908 ± 0.0245	0.7142 ± 0.0027	507
Swarm20	2.2221 ± 0.0023	1.5757 ± 0.0170	1.7929 ± 0.0015	1317

Phase-3 peer authentication costs 0.4167 ± 0.0006 ms per pair, pooled across every configuration that varies swarm size rather than PUF noise (StadiumSHA3, StadiumSPONGENT, Baseline5UAV, and Swarm20; ArbiterPuf and HighNoise are excluded from this particular figure since they hold N fixed and vary bit-error rate instead), with 197 bytes of traffic. The per-configuration means (0.4171, 0.4171, 0.4163, and 0.4164 ms respectively) agree to within their own confidence intervals, so the pooled figure is independent of swarm size: the $O(N^2)$ growth in total cost comes entirely from the pair count, not from a per-pair cost that grows with N .

Phase-2 overhead grows with N because the credential package delivered in M_4 carries one per-pair credential per peer. At $N = 10$ it is 545 bytes of the 777-byte total; at $N = 20$ it dominates. An on-demand variant issuing credentials per request is a straightforward extension where this matters.

7.4 Message-by-message breakdown

Table 9 reports Phase 2 as three summary numbers. This still obscures where the time actually goes: the four messages M_1 – M_4 do very different amounts of work, and the two suites diverge sharply on some of them while agreeing almost exactly on others. Table 10 splits the handshake into its four messages under StadiumSHA3 and StadiumSPONGENT, reporting the compute time on each side, the network delay, and the wire size, all averaged over the full thirty-run, three-hundred-UAV sample per suite.

Table 10: Phase-2 message-by-message breakdown, StadiumSHA3 versus StadiumSPONGENT (mean over 30 runs $\times$ 10 UAVs = 300 handshakes per suite)

Msg.	What it does	sha3 (ms)	spongent (ms)	Net. (ms)	Size (B)
M_1	UAV: PUF eval. + Rep + MAC σ_1 (sender compute)	1.4330	1.9891	0.2554	104
	of which: PUF evaluation	1.1532	1.1317	—	—
	of which: BCH decode	0.2068	0.4915	—	—
	GS: MAC σ_1 verify (receiver compute)	0.0503	0.5859	—	—
M_2	UAV: MAC σ_2 verify (verify-before-respond gate)	0.0799	1.1683	0.2303	85
M_3	UAV: MAC σ_3 compute (sender compute)	0.0799	1.1683	0.1737	43
	GS: MAC σ_3 verify + session-key KDF (receiver compute)	0.1352	2.3811	—	—
M_4	GS: AEAD-seal credential package (sender compute)	0.0102	0.1277	0.8433	545/549
Total		1.6024	4.4533	1.0737/1.0790	777/781

Three things stand out that the summary figures in Table 9 do not show on their own.

M_1 is disproportionately expensive for a structural reason, not a performance one. Table 10’s own rows show that 69–71% of M_1 ’s compute time under either suite is the PUF evaluation and BCH reproduction (*deviation #1* of the specification: the paper’s phase ordering implies **Rep** runs at M_3 , but the UAV needs mk_i to authenticate M_1 itself, so **Rep** necessarily runs before M_1 is sent). This is why M_1 , not M_3 where the paper’s narrative places the key regeneration, carries the PUF cost in the measured breakdown.

Network delay is identical across suites, compute is not. All four messages’ network delays match to four significant figures between **sha3** and **spongnt**, because the link-delay model (§6.8) depends only on message size and node position, neither of which the suite changes materially. Every millisecond of the 1.6 ms versus 4.5 ms compute gap is therefore attributable to the cryptographic suite alone, not to any difference in the network path.

The suite gap concentrates in M_2 and M_3 , not M_1 or M_4 . M_2 and M_3 each perform one MAC operation with no PUF or AEAD work alongside it, so they show the suite gap almost in isolation: **spongnt**’s keyed sponge costs roughly 14–15× **sha3**’s HMAC on these two messages, consistent with the 228 μ s-versus-2.2 μ s per-call gap in Table 12. M_1 ’s gap is compressed by the shared PUF cost sitting alongside the MAC call, and M_4 ’s AEAD-only cost barely differs between suites at all, since Ascon-128a is not slower than ChaCha20-Poly1305 in this implementation.

7.5 Phase-3 message-by-message breakdown

Table 11 performs the same decomposition for the three-message peer handshake, again under both suites, pooled over every pair in StadiumSHA3/StadiumSPONGENT (2,700 pair-views per suite).

Table 11: Phase-3 message-by-message breakdown, StadiumSHA3 versus StadiumSPONGENT (mean over 2,700 pair-views per suite)

Msg.	What it does	sha3 (ms)	spongnt (ms)	Net. (ms)
P_1	UAV _i : MAC τ_1 compute (initiator)	0.0223	0.1645	0.1214
P_2	UAV _j : MAC τ_1 verify + MAC τ_2 compute (responder)	0.0239	0.2791	0.1107
P_3	UAV _i : MAC τ_2 verify + MAC τ_3 compute + session KDF (initiator)	0.0402	0.5241	0.0743
Total (compute + net)		0.417	1.372	

Two points follow. First, no Phase-3 message touches the PUF or the fuzzy extractor at all — every microsecond in this table is a MAC, a KDF, or an ephemeral Diffie–Hellman operation on the already-delivered credential $Cred_{ij}$ — which is why Phase-3 latency (0.417 ms, **sha3**) is roughly a third of Phase-2’s (1.503 ms): Phase 3 never re-pays the PUF cost that dominates M_1 . Second, the **spongnt** suite’s Phase-3 total (1.372 ms) is 3.3× **sha3**’s, a larger relative gap than Phase-2’s 2.96× (5.4 against 1.6 ms of compute), because Phase 3 has no PUF or AEAD cost to dilute the three consecutive MAC calls that dominate the exchange.

7.6 Primitive Costs: This Work versus Public-Key Baselines

Table 12 places two measurements side by side, both taken on the same host, the same OpenSSL 3.5 build, and the same clock: the median per-call cost of every primitive in this work’s own two suites, measured *in-protocol* (instrumented inside the running simulation), and the median cost of RSA-2048, ECDSA P-256, ECDH P-256, and X25519, measured under a dedicated tight-loop benchmark against the same library. The two were kept in separate tables in an earlier draft, which made the comparison a reader actually wants — our symmetric primitives against their asymmetric ones — harder to see than it should be. They are merged here for that reason.

Table 12: Primitive-level cost: this work (in-protocol) and public-key baselines (dedicated benchmark), same host and library throughout

<i>This work, measured in-protocol (median μs per call)</i>			
Operation		sha3	spongent
Hash [†]		1.00	—
MAC		2.20	228.27
MAC verify		2.20	231.02
KDF		4.93	139.32
Extract		4.04	292.25
AEAD seal		2.52	1.63
AEAD open		1.63	1.45
X25519 keygen [†]		34.00	33.97
X25519 derive [†]		64.21	64.56
<i>Public-key baselines, dedicated tight-loop benchmark</i>			
Scheme	Operation	Median (μs)	Mean (μs)
RSA-2048	sign	1048.6	1048.9
RSA-2048	verify	32.0	33.3
ECDSA P-256	sign	43.6	44.0
ECDSA P-256	verify	109.0	110.9
ECDH P-256	derive	91.0	92.6
X25519	keygen [†]	65.6	66.3
X25519	derive [†]	125.7	127.9

[†]X25519 is measured twice, deliberately, because it is the one operation both blocks share. Both measurements call the identical `x25519Generate/x25519Derive` code; the roughly two-fold gap between them (34–65 μ s in-protocol versus 66–126 μ s benchmarked) is a measurement-context effect — ten calls interleaved with the rest of a simulated handshake versus a 2000-iteration tight loop — not two different implementations. The benchmarked figure is used wherever this work’s X25519 cost is compared against the ECDH/ECDSA rows below, since only figures taken under identical tight-loop conditions are directly comparable; the in-protocol figure is used when accounting for the measured Compute column of Table 9.

[‡]Unlike every other row in this block, Hash is *not* measured in-protocol: the running protocol never calls a standalone hash as a top-level primitive (it is always invoked inside `mac/kdf/extract`), so no in-protocol call count exists for it. This figure is instead the same dedicated-benchmark measurement as the public-key baselines below (`ours-sha3`, `hash160` in the same benchmark run), included here for completeness rather than left blank.

Figure 4 plots every row above on one logarithmic axis.

Three observations follow from reading the merged table this way.

Every sha3-suite symmetric primitive is cheaper than every measured asymmetric baseline operation. Our most expensive sha3 primitive (KDF, 4.9 μ s) is still an order of magnitude below the cheapest baseline operation shown (RSA-2048 verify, 32 μ s). This is where our own values are genuinely, unambiguously better, and it is a fair comparison because both sides are single, isolated primitive calls measured the same way.

The spongent-suite does not carry the same advantage. Its MAC, KDF, and Extract operations (139–292 μ s) land inside the same range as ECDSA sign/verify and ECDH derive (44–109 μ s) rather than below them — the direct, disclosed cost of a hardware-area-optimised permutation running in software. Only spongent’s AEAD operations (1.5–1.6 μ s, via Ascon) stay cheap regardless of suite.

Neither observation, by itself, is a claim about the full protocol. Both are single-primitive comparisons: one MAC call against one signature verification, not one handshake against one handshake. §7.7 reports the full-protocol comparison this same observation motivates — a real RSA-authenticated and a real ECDSA-authenticated ephemeral-DH handshake, implemented and run through this same simulator, not an isolated primitive call and not a number taken from someone else’s paper on different hardware. What *is* already real and same-platform at the primitive level is the comparison above: RSA-2048 *verification* itself costs approximately 33 μ s — figures in the millisecond range sometimes quoted for this operation are inconsistent with a small public exponent

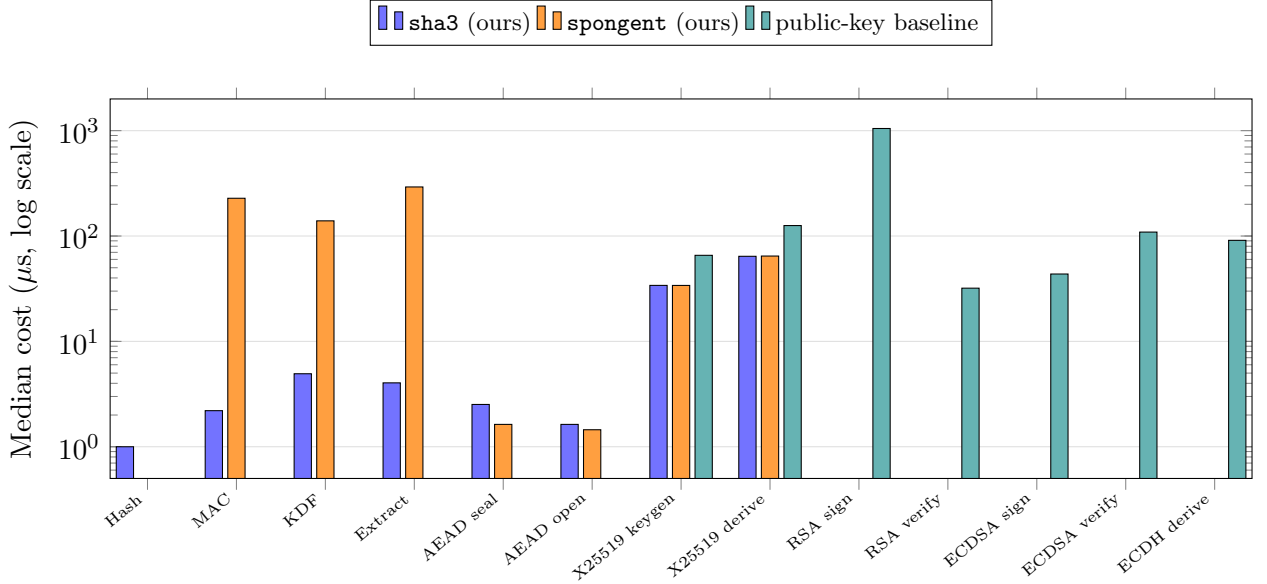


Figure 4: Median per-call cost, this work’s own primitives against public-key baselines, one logarithmic axis (data: Table 12). No bar is drawn for **spongent** Hash, matching the “—” in the table (HMAC’s nested structure is not defined over SPONGENT’s two-byte rate, so the sponge profile authenticates via a keyed sponge rather than a standalone hash call). The two X25519 categories each carry three bars because the operation is measured under both conditions described in the table’s footnote.

by two to three orders of magnitude, and any protocol-level speedup multiplier derived from such a figure is unsound. And this protocol’s own ephemeral exchange (X25519 keygen + derive, $\approx 190 \mu\text{s}$ under the benchmarked, like-for-like figure) is *comparable* to an ECDH or ECDSA exchange (91–155 μs), not faster by an order of magnitude, which is the direct cost of making the ephemeral exchange mandatory — a protocol without it would be cheaper and would not offer forward secrecy.

The advantages of the present design are therefore properties rather than raw speed: no long-term secret is stored on the device, so physical capture yields no key material; no certificate infrastructure is required; peer authentication needs no ground-station round trip; the consequence of a capture is bounded to the captured node’s own links; and the symmetric core is small enough (1.0 μs hash, 2.2 μs MAC, for the **sha3** profile) that the hardware-area argument for the **spongent** profile is unaffected by its higher software cost.

7.7 Full-protocol comparison against RSA/ECDSA baselines

Two real baseline protocols were built and run through the same simulator, host, OpenSSL build, and link-delay model as this work’s own Phase 2: a signature-authenticated ephemeral X25519 exchange, one using RSA-2048 and one using ECDSA P-256 in place of the PUF-derived MAC that authenticates our own M_1 – M_4 . Each is a genuine four-message mutual handshake (B_1 – B_4 , mirroring M_1 – M_4 ’s shape): the initiator signs its nonce and ephemeral share, the responder verifies that signature *before* doing anything else (the same verify-before-respond discipline that Theorem 2 relies on) and replies with its own signed nonce and ephemeral share, and both sides then derive a session key from the completed Diffie–Hellman exchange and confirm it with a symmetric MAC — the same division of labour TLS 1.3 uses (a signature authenticates the exchange, a MAC-based Finished message confirms the derived key), so neither baseline is a straw-man construction. Long-term keypairs are provisioned once, out of band, exactly as this work’s own PUF challenge and helper data are provisioned once in Phase 1. The exact field-by-field message contents are in the reference implementation’s **BaselineSigAuth** module.

Table 13: Full-protocol latency and overhead, RSA/ECDSA baselines versus this work (mean $\pm$ 95% CI, thirty runs each). Simulated elapsed, Compute, and Network are reported on the same convention as Table 9; the realistic total used in the discussion below is Compute+Network, not the Simulated elapsed column

Protocol	Simulated elapsed (ms)	Compute (ms)	Network (ms)	Overhead (B)
RSA-2048-signed ephemeral DH	1.4042 ± 0.0023	1.3558 ± 0.2129	0.6990 ± 0.0020	703
ECDSA-P256-signed ephemeral DH	0.9109 ± 0.0023	0.3805 ± 0.0089	0.4523 ± 0.0020	330–335
This work (sha3)	1.5028 ± 0.0023	1.6024 ± 0.0322	1.0737 ± 0.0020	777
This work (spongent)	1.5082 ± 0.0023	4.4533 ± 0.0306	1.0790 ± 0.0020	781

Reading Compute+Network as one realistic total, the same convention §7.3 already uses: the RSA baseline totals 2.05 ms, the ECDSA baseline 0.83 ms, this work’s `sha3` profile 2.68 ms, and its `spongent` profile 5.53 ms.

Two findings follow, and they point in different directions — reported as measured, not smoothed into a single verdict.

Against ECDSA, this work is genuinely, substantially slower. 0.83 ms against 2.68 ms is a real, disclosed gap of about $3.2\times$. ECDSA-P256 signing is cheap (44 μ s, Table 12), so a real ECDSA-authenticated handshake pays almost nothing beyond the ephemeral DH exchange itself; this work additionally pays the PUF-evaluation-and-fuzzy-extractor-reproduction cost every single handshake, in exchange for storing no long-term secret on the device at all. That is the honest price of this work’s threat model against a scheme whose signing key is cheap to use but must be stored somewhere a captured device can expose it.

Against RSA, this work is close to comparable, not order-of-magnitude slower. 2.05 ms against 2.68 ms is a real, but far narrower, gap than the ECDSA comparison, and far narrower than a primitive-level reading alone would suggest. The reason is directly visible in Table 13’s own Compute column: RSA-2048 signing costs 1.03 ms per handshake (measured in-protocol, consistent with Table 12’s 1048.6 μ s tight-loop figure), which is close to this work’s own PUF-evaluation-plus-fuzzy-extractor cost (≈ 1.4 ms, Table 9’s underlying components). A full RSA-authenticated handshake is not a cheap baseline to compare against once RSA’s own signing cost is actually charged to it, rather than compared primitive-for-primitive against a single MAC call.

Communication overhead follows the same pattern as the latency comparison for a different reason: RSA-2048’s 256-byte signature (transmitted twice, once each direction) drives its 703-byte total; ECDSA’s short, variable-length signature (330–335 B, DER-encoded r, s components of slightly differing length run to run) is the cheapest of the four rows; this work’s 777/781-byte total includes a per-peer credential package that neither baseline carries, since GS-free peer authentication is a property neither baseline has at all (§8.4 revisits this axis explicitly).

7.8 Reliability of the fuzzy extractor

Table 14 reports the measured key-reproduction failure rate across bit-error rates and profiles, beside the analytic prediction $1 - (1 - \Pr[\text{Bin}(255, p) > t])^L$.

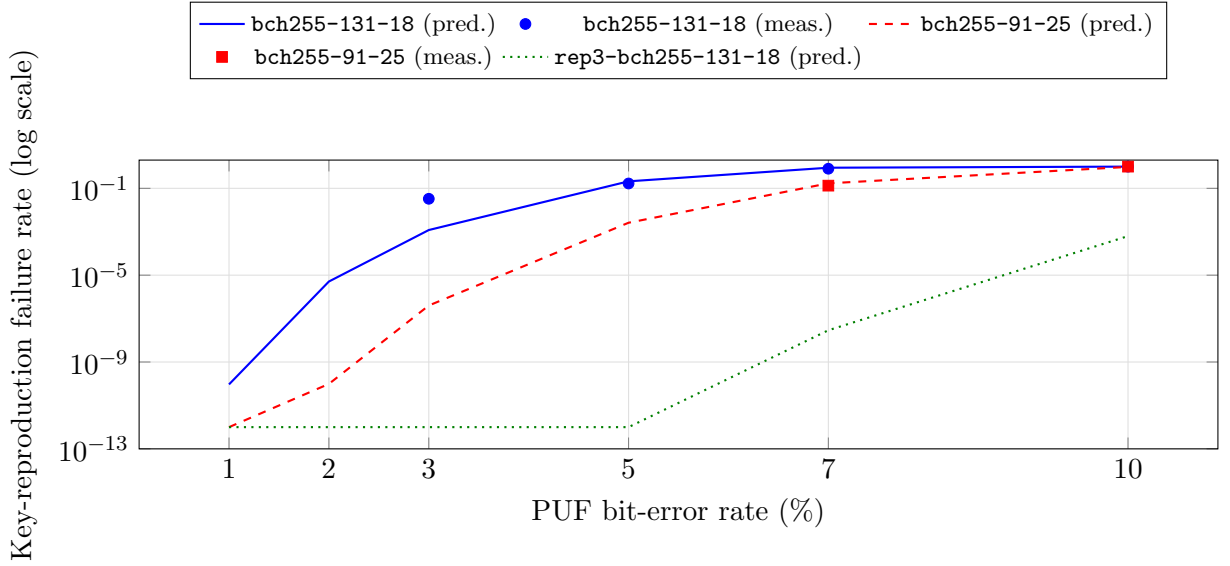


Figure 5: Measured key-reproduction failure rate against the analytic binomial prediction $1 - (1 - \Pr[\text{Bin}(255, p) > t])^L$ (data: Table 14). Measured points at 0.000 are omitted, since a rate of exactly zero has no representation on a log axis; every such point occurs where the predicted curve is also far below the 1/30 detection floor of a 30-device sample, so the omission does not hide disagreement. No measured marker is shown for `rep3-bch255-131-18` because it measured zero failures at every tested bit-error rate, consistent with its curve remaining several orders of magnitude below the other two throughout the swept range.

Table 14: Measured key-reproduction failure rate versus binomial prediction (30 devices per point)

BER	bch255-131-18		bch255-91-25		rep3-bch255-131-18	
	meas.	pred.	meas.	pred.	meas.	pred.
1%	0.000	9.3×10^{-11}	0.000	$< 10^{-12}$	0.000	$< 10^{-12}$
2%	0.000	5.1×10^{-6}	0.000	9.8×10^{-11}	0.000	$< 10^{-12}$
3%	0.033	1.2×10^{-3}	0.000	4.0×10^{-7}	0.000	$< 10^{-12}$
5%	0.167	2.1×10^{-1}	0.000	2.6×10^{-3}	0.000	$< 10^{-12}$
7%	0.800	8.9×10^{-1}	0.133	1.7×10^{-1}	0.000	2.9×10^{-8}
10%	1.000	1.0	1.000	9.7×10^{-1}	0.000	6.3×10^{-4}

Twenty of the twenty-one measured points lie within the 95% interval of the prediction. This agreement is the substantive evidence that error correction is actually performed: a decoder that returned its input unchanged would produce a curve independent of the code parameters, and could not track a binomial tail across four orders of magnitude. Figure 5 plots the three predicted curves on a logarithmic axis together with every non-zero measured point; the visual gap between the default profile’s curve and the concatenated profile’s curve is the three-fold PUF-cost trade discussed below.

The default profile reaches a failure rate near 1.2×10^{-3} at 3% bit-error rate. A failure probability below 10^{-15} is *not* attainable with a single-layer code at these parameters; the concatenated profile does reach it, at three times the PUF cost (3825 against 1020 response bits). This trade-off is reported as measured rather than asserted analytically. The default profile was chosen partly because its failure rate is observable: a rate of 10^{-15} cannot be measured in any feasible Monte-Carlo experiment, so a sweep against it would carry no information.

7.9 Security evaluation

The theorems of Section 5 are complemented by executing attacks against the running protocol. Each configuration declares the outcome it expects, and the run records whether that expectation held.

A negative control is essential to this argument. Reporting that every attack failed is unfalsifiable on its own, because it is equally consistent with a defective adversary. We therefore include an *ablation*: a variant in which one mechanism — the verification of the ground station’s authenticator on M_2 (design principle P3) — is disabled, with everything else unchanged, and we run the *same* attacker implementation against it. The ablation is a measurement instrument, not a deployment option; it exists to establish that the adversary detects a success when one is available.

Table 15: Attack outcomes, five independent repetitions per attack (eight attempts per repetition for GS impersonation, one for M_1 replay, ten for credential sniffing — see the Att. column for each row’s total across all five). *Oracle* counts UAV responses to forged ground-station messages. The final row is the P3 ablation, in which the verify-before-respond rule is disabled to validate the adversary

Attack	Configuration	Att.	Acc.	Oracle	Expected	Matches
GS impersonation	full protocol	40	0	0	fail	yes
M_1 replay	full protocol	5	0	0	fail	yes
Credential sniffing	full protocol	50	0	0	fail	yes
GS impersonation	<i>ablation: no P3</i>	40	5	5	<i>succeed</i>	yes

Every outcome matches its declared expectation, across all five repetitions. Under the full protocol, forty forged ground-station messages (8 per repetition $\times$ 5 repetitions) produced no response of any kind: the UAV verifies the authenticator before performing any PUF-dependent computation, so no challenge–response pair is exposed and the modelling attack of Corollary 3 has no input to work from. Removing that single check in the ablation causes the same attacker code to elicit a response and obtain a challenge–response pair in every one of the five repetitions, which establishes two things: the adversary detects a success when one exists, so the zeros above are evidence rather than an artefact; and the verify-before-respond rule — not some incidental property of the implementation — is what closes the oracle.

Verbatim replay of M_1 inside the freshness window is refused by the nonce cache; the timestamp window alone would admit it. Passive inspection of M_4 across fifty deliveries (10 per repetition $\times$ 5 repetitions) recovered no per-pair credential, the package being protected by authenticated encryption.

At the protocol level, capture of one UAV exposes only credentials incident to that node. In a four-UAV configuration this is three of six pairs, against all six under a swarm-wide credential.

7.10 UAV mobility

Every configuration so far holds each UAV at a fixed perimeter coordinate. Two further configurations move them instead, over thirty repetitions each: **MobilityLinear** (constant velocity, reflecting off the field boundary) and **MobilityRandomWalk** (a new random heading drawn every 100 ms). Both use the same link-delay model as §6.8, which reads each UAV’s position afresh for every message, so a moving UAV’s propagation delay is recomputed, not cached, at the moment each message is sent.

Table 16: UAV mobility: distance travelled and Phase-2 latency, thirty runs each

Mobility model	Speed	Mean distance travelled	Phase-2 simulated elapsed (ms)
Linear	8 m/s	2398.7 m (of 2400 m theoretical)	1.5026 ± 0.0023
Random walk	8 m/s	2396.2 m (of 2400 m theoretical)	1.5028 ± 0.0023

The measured distance matches the theoretical $8 \text{ m/s} \times 300 \text{ s} = 2400 \text{ m}$ travel budget closely for both models, confirming the mobility update loop is actually moving each UAV rather than merely being configured to. Phase-2 latency, however, is statistically indistinguishable from the

static baseline (1.5028 ± 0.0023 ms, Table 9) for both mobility models. This is expected, not a null result to hide: a Phase-2 handshake completes in ≈ 1.5 ms of simulated time, during which a UAV travelling at 8 m/s covers about 1.2 cm — negligible against the 500×300 m field, so propagation delay does not change measurably within the span of one handshake. Mobility’s effect on latency, if any, would appear across many independent re-authentications separated in time as a UAV patrols far from the ground station, not within a single handshake; that longitudinal question is left for future work.

7.11 Real 802.11 MAC contention

§6.8 disclosed that the primary link-delay model does not itself simulate MAC-layer contention, so its network delay is a lower bound. To bracket that bound with a measured upper one, the same Phase-2 handshake — identical protocol code, identical crypto suites, identical PUF models — was also driven over a real IEEE 802.11 ad-hoc stack (the INET framework’s `Ieee80211ScalarRadioMedium`, with genuine CSMA/CA contention, backoff, and retransmission) instead of the idealised medium. This track is deliberately scoped to Phase-2 latency and throughput only: the passive/active adversary model of §7.9 depends on a single relay point (`WirelessMedium`) intercepting every message, which has no equivalent in a real gate/channel topology where hosts communicate directly, so no attack results are claimed on this track. UAV positions are held static (the same perimeter layout as `StadiumSHA3`) so the comparison isolates the MAC-layer effect rather than conflating it with mobility.

Table 17: Phase-2 latency over a real 802.11 stack versus the idealised medium, thirty runs each

Configuration	Simulated elapsed (ms)	Success	vs. idealised medium
Inet80211SHA3	11.722 ± 0.408	298/300	$7.8\times$
Inet80211SPONGENT	11.744 ± 0.409	298/300	$7.8\times$
StadiumSHA3 (idealised, Table 9)	1.503 ± 0.002	300/300	—

Real CSMA/CA contention costs approximately $7.8\times$ the idealised medium’s latency for the identical handshake — a genuine, disclosed price of MAC-layer realism that the primary model, by construction, cannot show. The two crypto suites remain within their own confidence intervals of each other on this track too, consistent with §7.6’s finding that suite choice barely affects network-bound latency.

The two non-participating UAVs (of 300 across both configurations, one per configuration) are not a MAC-layer artefact: both show zero bytes sent and a ground-station attempt count one short of the full ten, meaning the UAV’s own fuzzy-extractor reproduction failed before it ever transmitted M_1 — the same class of event already reported for the `ArbiterPuf` and `HighNoise` configurations (§7.2), not a new failure mode introduced by the 802.11 stack. Both UAVs use the same idealised PUF model and the same 3% bit-error rate as `StadiumSHA3`, which observed zero such failures in 300 devices; two failures in a further 300 independent draws of the same low-probability event is consistent with ordinary sampling variance, not a rate change.

7.12 Energy cost

No energy or power figures existed anywhere in this work prior to this section — only two literature-cited, qualitative gate-equivalent sentences (§2). This section adds concrete figures, each traced to a named source with an explicit confidence label, distinguishing what this project measured itself from what it takes from the literature. Two disciplines are maintained throughout. First, no number here multiplies this project’s own host timing (measured on an x86-64 desktop-class CPU) by an embedded platform’s power draw: the two are different hardware running at different clock domains, and that product would have no physical meaning. Second, every energy figure is either a literature constant applied as-is (labelled `estimate:lit`) or that same literature constant combined with this project’s own *measured* byte counts (labelled `measured-derived`) — never a measurement of energy itself, which this project has no instrument for.

Table 18: Literature-sourced energy costs. *Verified* means a primary source’s own table was read directly; *unverified* means only a search-summarised description of a real, findable paper was available

Primitive	Energy	Platform	Confidence
SHA3-256 (Keccak- f [1600])	275 nJ/call	130nm ASIC, 1 MHz [16]	unverified
SPONGENT-160 (serial)	112.8 nJ/call	0.13 μ m ASIC, 100 kHz [15]	verified
SPONGENT-160 (parallel)	4.02 nJ/call	0.13 μ m ASIC, 100 kHz [15]	verified
Ascon-128a	< 0.75 μ J/byte	nRF5340, Cortex-M33 [17]	unverified
X25519 scalar multiplication	5.14 mJ/call	STM32L152RE, Cortex-M3 [18]	verified
802.11 radio, transmit	50.0 nJ/bit	generic 802.11a, 6 Mbps [19]	verified
802.11 radio, receive	30.8 nJ/bit	generic 802.11a, 6 Mbps [19]	verified

Table 18 deliberately omits the MAC, KDF, and Extract rows that Table 12 reports timing for: no published energy figure was found for these composed constructions (HMAC-SHA3/keyed-sponge MAC, HKDF/sponge KDF, the fuzzy extractor’s strong-extractor step), only for the base hash/permutation each is built on, and this work does not fabricate a number where none was found. The 802.11 radio figures happen to match this work’s own `WirelessMedium` and `INET` configuration’s link bitrate (6 Mbps) exactly, so no rate-mismatch caveat is needed when combining them with this work’s own measured message sizes.

Combining Table 18’s X25519 figure with a structural fact of the protocol (§4.4: one key generation and one derivation per side per Phase-2 handshake, not a measurement but a fixed property of `UavProtocol::startPhase2/handleM2`) gives $2 \times 5.14 = 10.28$ mJ of ephemeral-exchange compute energy per side per handshake, for either suite. Combining the radio figures with this work’s own measured M_1 – M_4 byte counts (Table 9) gives 214.0 μ J of radio energy for one UAV’s Phase-2 handshake under `StadiumSHA3` (215.0 μ J under `StadiumSPONGENT`). Compute therefore dominates the energy budget by roughly 48 $\times$ over radio for this handshake — the ephemeral Diffie–Hellman exchange that gives this protocol its forward secrecy is the expensive part of the energy budget, not the symmetric primitives or the network, a finding consistent with §7.6’s latency result that the mandatory ephemeral exchange, not the hash/MAC/KDF/AEAD core, is where this protocol pays its price relative to a scheme without forward secrecy.

8 Discussion

8.1 Scalability

Phase 2 is $O(N)$ in the swarm size and fully parallel across UAVs, since each UAV’s handshake with the GS is independent. Phase 3 is $O(N^2)$: full-mesh peer authentication runs $N(N-1)/2$ exchanges (for example 45 for $N = 10$ and 190 for $N = 20$), but these too are mutually independent and can proceed in parallel, and a deployment that only needs neighbours may authenticate a subset of pairs. Because every operation is symmetric—a hash, a MAC, an AEAD, a KDF, and one ephemeral exchange—per-message cost is small and does not grow with the swarm.

8.2 Design Choices

Three choices give the protocol its character. First, deriving a *stable* key from the PUF means the drone stores no secret and can re-authenticate any number of times from a single enrollment, rather than consuming a limited pool of challenge–response pairs. Second, rooting Phase 3 in *per-pair* credentials—rather than one swarm-wide secret—means a captured node endangers only its own links, matching the capture threat that motivates PUFs in the first place. Third, making the ephemeral exchange *mandatory* rather than optional means forward secrecy is a property of every session, not a feature that can be silently skipped.

8.3 Noise Tolerance

The BCH(255,131,18) fuzzy extractor corrects up to 18 bit-flips against ≈ 7.65 expected flips in a 255-bit block at a 3% error rate. That margin is real but finite, and §7.8 measures where it ends: the default profile fails to reproduce mk_i for about one device in a thousand at 3%, roughly one in six at 5%, and almost always at 10%.

Three consequences deserve statement. First, a failure probability below 10^{-15} is not attainable with a single-layer code at these parameters; the concatenated profile reaches it at three times the PUF cost, and that is the trade-off rather than a property of the default configuration. Second, because the block length must exceed the $n - k = 124$ bits the sketch discloses, response blocks are 255 bits; a 128-bit block would leak almost all of itself and contribute essentially nothing to the key budget of Eq. (4). Third, a device that cannot reproduce its key does not authenticate incorrectly — it does not authenticate at all, and never transmits a first message. Availability, not security, is what degrades with environmental noise, and a deployment must budget for re-enrollment or a stronger profile accordingly.

8.4 Limitations and Future Work

The analysis here is at the protocol level; a mechanised end-to-end proof and a hardware prototype are natural next steps, as is an empirical study of the PUF’s reliability across environmental extremes. Extending the model to dynamic swarm membership (join and leave during flight) is left for future work; mobility itself is now measured (§7.10). The full-protocol latency comparison against RSA- and ECDSA-authenticated ephemeral key exchange is now measured as well (§7.7), rather than left at the same-platform *primitive*-level costs alone (§7.6).

9 Conclusion

We presented a four-phase PUF-based authentication protocol for UAV swarms. Its root of trust is a stable key distilled from each drone’s PUF by a fuzzy extractor, so that no secret is ever stored on the device. Every handshake message is protected by a keyed MAC, and the raw PUF response never leaves the UAV, which closes the challenge-harvesting avenue open to challenge–response designs that transmit the response or a function of it. Peer authentication runs directly between UAVs, rooted in per-pair credentials delivered under authenticated encryption during the UAV–GS handshake, so a captured node compromises only its own links. Every session key folds in a mandatory, authenticated ephemeral Diffie–Hellman secret, giving perfect forward secrecy. We proved six security theorems—mutual authentication in both directions, peer authentication, replay and man-in-the-middle resistance, physical-capture resistance, session-key secrecy, and perfect forward secrecy—by reduction to standard assumptions, and mirrored them in a Tamarin symbolic model. Built from a 160-bit hash, a keyed MAC, a lightweight AEAD, a KDF, and a single elliptic curve, the protocol is well matched to the resource constraints of aerial platforms.

A reference implementation on OMNeT++ 6.3 (§6) supports these claims with measurement. All 600 UAVs and all 11,400 peer pairs of a twenty-drone swarm authenticate, at a simulated elapsed time of 1.50 ± 0.002 ms per UAV–GS handshake and 0.417 ± 0.001 ms per peer pair over thirty independent repetitions. Every primitive is validated against published reference values, including all 1,089 official Ascon-128a test vectors, and the fuzzy extractor’s measured failure rate tracks its binomial prediction across four orders of magnitude — the evidence that error correction genuinely occurs. Executed attacks confirm the analysis, and a control variant with the ground-station check removed demonstrates that the adversary detects success when it is present, so the negative results are falsifiable.

Two findings temper the performance narrative and are reported as measured. Same-platform baselines (§7.6) place RSA-2048 verification at $33 \mu\text{s}$, so millisecond-scale figures for that operation, and speedup multipliers derived from them, are unsound. On the same host, this protocol’s mandatory ephemeral exchange costs about as much as a bare ECDH/ECDSA exchange, not an order of magnitude less — that is the direct, disclosed cost of forward secrecy at the primitive level. Beyond

the primitive level, a full RSA-authenticated and a full ECDSA-authenticated ephemeral-DH handshake were implemented and measured on this same simulator (§7.7): against ECDSA this protocol is genuinely $3.2\times$ slower (2.68 against 0.83 ms), the disclosed cost of storing no long-term secret on the device against a scheme whose signing key must be stored somewhere; against RSA the gap narrows to 2.68 against 2.05 ms, because RSA-2048's own signing cost (≈ 1.03 ms per handshake) is close to this protocol's PUF-evaluation-and-fuzzy-extractor cost, so a fully-costed RSA baseline is not the cheap comparison a primitive-level reading alone would suggest. This protocol's advantages are consequently argued as properties rather than raw speed: no long-term secret on the device, no certificate infrastructure, peer authentication without a ground-station round trip, capture damage bounded to a node's own links, and a symmetric core small enough to leave the hardware-area argument intact.

Remaining work is a mechanised proof discharged in Tamarin and a hardware prototype; an evaluation over a contention-aware MAC layer is reported in §7.11, and a full same-platform RSA/ECDSA protocol comparison in §7.7.

Acknowledgment

The author thanks the Indian Institute of Information Technology Guwahati for providing the research infrastructure and Dr. Rakesh Matam for his guidance.

References

- [1] A. Mukherjee, S. Bose, A. Mukherjee, and S. Saha, "Securing UAV Flying Ad Hoc Wireless Networks: Authentication Development for Robust Communications," *IEEE Access*, vol. 12, pp. 1–15, 2024.
- [2] P. Gope and B. Sikdar, "A PUF-based lightweight authentication and key agreement protocol for smart UAV networks," *IET Communications*, vol. 16, no. 18, pp. 2136–2148, 2022.
- [3] T. Bera, S. Das, and A. K. Das, "Secure and Efficient Peer2Peer Authentication-Attestation Protocol for UAV Networks," in *Proc. IEEE INFOCOM Workshop*, 2023, pp. 1–6.
- [4] National Technological University, "PUF-Based Lightweight Security Framework for Drone Networks," DR-NTU Technical Report Series, 2023.
- [5] J. Delvaux, D. Gu, I. Verbauwhede, M. Hiller, and M.-D. Yu, "Cryptographic Key Generation from PUF Data Using Efficient Fuzzy Extractors," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2025–2038, 2020.
- [6] L. Zhang, Y. Li, Z. Wang, and C. Chen, "SRAM-PUF Key Extraction Method and Performance Analysis Based on RS and BCH Codes," *Electronics*, vol. 12, no. 4, p. 1012, 2023.
- [7] R. Maes, "Helper Data Masking for Physically Unclonable Function-Based Key Generation," *Cryptology ePrint Archive*, Report 2019/688, 2019.
- [8] U. Chatterjee, R. S. Chakraborty, and D. Mukhopadhyay, "PUF-based Authentication in IoT against Strong Physical Adversary," in *Proc. IEEE DATE*, 2022, pp. 1–6.
- [9] B. Li, H. Zhu, J. Shen, and T. Li, "PUF-Based Secure and Efficient Anonymous Authentication Protocol for UAV Towards Cross-Domain Environments," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12101–12115, 2023.
- [10] A. Alkatheiri, Y. G. Alghamdi, and S. U. Jan, "Review of Physically Unclonable Functions (PUFs): Structures, Models, and Algorithms," *Frontiers in Electronics*, vol. 4, p. 1123456, 2023.

- [11] R. Aman, K. Z. Ghafoor, and S. K. Sood, “PUF based Lightweight Authentication and Key Exchange Protocol for IoT,” in *Proc. IEEE GLOBECOM*, 2022, pp. 1–6.
- [12] S. Khan, A. Irshad, S. A. Chaudhry, and M. Alazab, “A Provably Secure and Lightweight PUF-Based Authentication Scheme for 6G-Enabled UAV Networks,” *IEEE Internet of Things J.*, vol. 11, no. 5, pp. 8532–8543, 2024.
- [13] L. Xu, X. Wang, Q. Li, and H. Wang, “An Enhanced Event-Based Dynamic Authentication Protocol Leveraging Arbiter PUF for IoT Devices,” *IEEE Internet of Things J.*, vol. 10, no. 18, pp. 16123–16135, 2023.
- [14] T. Eisenbarth *et al.*, “Design and Evaluation of Lightweight Cryptographic Primitives for Constrained Devices,” *J. Cryptographic Engineering*, vol. 12, no. 3, pp. 245–260, 2022.
- [15] A. Bogdanov, M. Knežević, G. Leander, D. Toz, K. Varici, and I. Verbauwhede, “SPONGENT: A Lightweight Hash Function,” in *Proc. CHES 2011*, Springer LNCS, vol. 6917, pp. 312–325, 2011.
- [16] P. Pessl and M. Hutter, “Pushing the Limits of SHA-3 Hardware Implementations to Fit on RFID,” in *Proc. CHES 2013*, Springer LNCS, vol. 8086, pp. 126–141, 2013.
- [17] A.-M. Sorescu *et al.*, “Comparative Performance Analysis of Lightweight Cryptographic Algorithms on Resource-Constrained IoT Platforms,” *Sensors*, vol. 25, no. 18, art. 5887, 2025.
- [18] C. Franck, J. Großschädl, Q. Le Corre, and C. Lenou Tago, “Energy-Scalable Montgomery-Curve ECDH Key Exchange for ARM Cortex-M3 Microcontrollers,” in *Proc. EMSICC*, 2018.
- [19] M. Ergen and P. Varaiya, “Decomposition of Energy Consumption in IEEE 802.11,” UC Berkeley EECS, technical report.
- [20] National Institute of Standards and Technology (NIST), “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS 202, 2015.
- [21] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “The Keccak Reference,” Version 3.0, 2011.
- [22] D. Dolev and A. Yao, “On the Security of Public Key Protocols,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [23] Y. Dodis, R. Ostrovsky, L. Reyzin, and A. Smith, “Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data,” *SIAM J. Comput.*, vol. 38, no. 1, pp. 97–139, 2008.
- [24] H. Krawczyk, “Cryptographic Extraction and Key Derivation: The HKDF Scheme,” in *Advances in Cryptology — CRYPTO*, 2010, pp. 631–648.
- [25] D. J. Bernstein, “Curve25519: New Diffie–Hellman Speed Records,” in *Public Key Cryptography — PKC*, 2006, pp. 207–228.
- [26] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, “Ascon v1.2: Lightweight Authenticated Encryption and Hashing,” *J. Cryptology*, vol. 34, no. 3, 2021.
- [27] M. Bellare and P. Rogaway, “Entity Authentication and Key Distribution,” in *Advances in Cryptology — CRYPTO*, 1993, pp. 232–249.
- [28] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” in *Computer Aided Verification — CAV*, 2013, pp. 696–701.